1.3.5 Đo Cao Độ Tiếng Nói (F_0 / Pitch): Từ Bản Chất Praat Đến Vi Kiến Trúc FPGA

► MỤC TIÊU HỌC TẬP

Mục 1.3 đã biến 400 mẫu áp suất không khí thành 80 con số Log-Mel để phục vụ các mạng nơ-ron nhận dạng từ khóa. Nhưng các con số đó hoàn toàn câm lặng trước một đặc trưng sinh lý học cơ bản của con người: **Cao độ (Pitch / Tần số cơ bản F_0)** — tần số rung của dây thanh quản. Mục này giải thích tại sao không thể dùng phổ STFT để đo F_0, bóc tách bản chất giải thuật tự tương quan thời gian theo tinh thần mã nguồn Praat (`Sound_to_Pitch.cpp`, Boersma 1993), tối giản hóa các phép toán giải tích phức tạp thành các khối phần cứng thân thiện (chỉ dùng nhân-cộng và so sánh), và định lượng tài nguyên silicon trên kit FPGA DE10-Standard (Intel Cyclone V).

0. Tại sao cần một luồng xử lý riêng cho Cao Độ (F_0 Stream)?

Trong chuỗi xử lý tiếng nói ở biên (Edge Voice AI), các kỹ sư thường mắc một ngộ nhận kinh điển: nghĩ rằng vector Log-Mel Spectrogram ở Mục 1.3 đã chứa đủ mọi thông tin để giải quyết mọi bài toán.

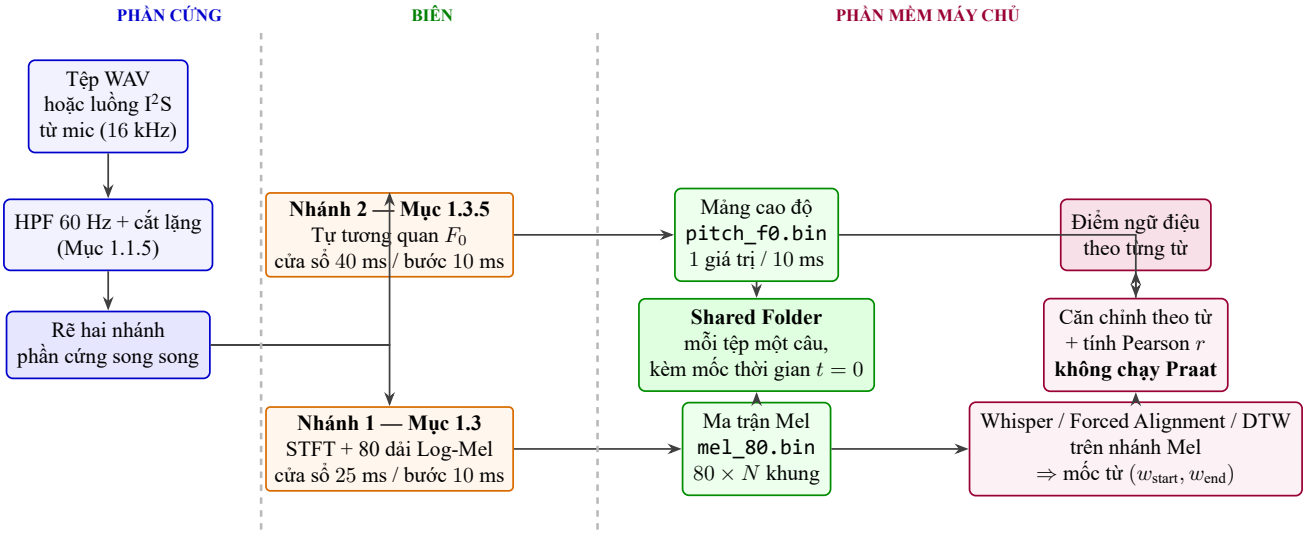
Thực tế, **Ngân hàng bộ lọc Mel sinh ra là để xóa bỏ cao độ**:

- 80 dải lọc tam giác Mel gom và tích phân năng lượng trên các dải tần số rộng hàng trăm Hertz nhằm mô phỏng hình thái của **bao phổ (spectral envelope)** — tức hình dáng ống thanh âm (vocal tract) đại diện cho các nguyên âm.
- Quá trình gom bin đó cố tình san phẳng các vạch hài âm thanh quản mảnh dẻ.
- Trong các bài toán ngữ điệu, phát hiện cảm xúc, nhận diện thanh điệu tiếng Việt (ngã, hỏi, sắc, huyền, nặng, ngang), hoặc đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như **Praat**, hệ thống bắt buộc phải đo chính xác từng Hertz của tần số cơ bản $F_0 = 1/T_0$ (với T_0 là chu kỳ giữa hai lần mở-đóng liên tiếp của dây thanh quản).

Do đó, một hệ thống âm thanh biên hoàn chỉnh bắt buộc phải tách thành hai luồng song song: **Luồng nhận dạng âm học (Log-Mel Frontend)** và **Luồng đo cao độ (F_0 / Pitch Stream)**.

0.1. Kiến trúc triển khai thực tế: FPGA Edge Node ↔ Shared Folder ↔ Server

Hai luồng vừa nêu không chỉ là hai khối thuật toán, mà là hai nhánh của một hệ thống triển khai có ranh giới phần cứng/phần mềm rõ ràng. Hình 1.3.5a vẽ ranh giới đó.



Hình 1.3.5a: Kiến trúc triển khai tổng thể: FPGA Edge Node tính cả hai nhánh đặc trưng và ghi ra thư mục dùng chung; máy chủ chỉ còn nhiệm vụ nhận dạng, căn chỉnh và chấm điểm.

Đọc Hình 1.3.5a (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Cột trái — Phần cứng (FPGA DE10-Standard):** Mọi phép biến đổi tín hiệu đều nằm ở đây. Tệp WAV hoặc luồng I^2S đi qua HPF 60 Hz và bộ cắt lặng của Mục 1.1.5 trước khi được rẽ vào hai nhánh chạy song song trên cùng một chip. Hai nhánh không tranh chấp tài nguyên vì nhánh Mel dùng bộ nhân FFT còn nhánh Pitch dùng một bộ MAC đơn, như đã hạch toán ở Mục 5.
2. **Nhánh 1 (Mục 1.3):** Sinh ma trận 80 dải Log-Mel với cửa sổ 25 ms và bước 10 ms. Đầu ra này là **đầu vào sẵn sàng** cho bộ mã hóa Whisper, hoặc được ghi ngược thành tệp WAV 16 kHz đã lọc sạch để gọi API Whisper từ xa.
3. **Nhánh 2 (Mục 1.3.5):** Sinh mảng cao độ một giá trị mỗi 10 ms. Đây là toàn bộ những gì máy chủ cần cho ngữ điệu, và nó đã được tính xong **trước khi** dữ liệu rời khỏi bo mạch.
4. **Cột giữa — Thư mục dùng chung:** Là ranh giới hợp đồng giữa hai thế giới. FPGA chỉ ghi, máy chủ chỉ đọc, nên không bên nào phải biết định dạng bộ nhớ của bên kia. Mỗi câu một cặp tệp, kèm quy ước chung rằng mốc $t = 0$ là mẫu đầu tiên sau khi cắt lặng.
5. **Cột phải — Máy chủ:** Chạy mô hình nhận dạng hoặc căn chỉnh trên nhánh Mel để lấy mốc thời gian từng từ, rồi ghép trực tiếp mảng `pitch_f0.bin` vào các mốc đó để tính Pearson r. **Máy chủ không chạy Praat**, vì phép đo cao độ đã hoàn tất trên FPGA.

Vì sao ranh giới lại đặt ở đây. Có một lý do kỹ thuật chứ không phải thẩm mỹ. Phép đo cao độ là phép tính **cố định** về mặt cấu trúc: cùng một khung 640 mẫu luôn cho cùng một kết quả, không phụ thuộc mô hình ngôn ngữ, không cần cập nhật khi đổi phiên bản Whisper. Phép nhận dạng thì ngược lại: nó thay đổi mỗi lần có mô hình mới. Đặt phép tính bất biến lên silicon và phép tính hay thay đổi lên máy chủ nghĩa là nâng cấp mô hình không phải đụng đến bitstream, và một kit FPGA rẻ có thể phục vụ nhiều phiên bản mô hình khác nhau theo thời gian. Đây cũng là lý do đường biên trong hình được vẽ bằng nét đứt đậm: nó là ranh giới **hợp đồng dữ liệu**, không phải ranh giới vật lý của một sản phẩm cụ thể.

1. Phương pháp thô sơ bị bác bỏ: Dùng STFT và Argmax trên phổ tần số

Phương pháp trực giác ngây thơ nhất là: *Mục 1.3 đã tính sẵn biến đổi Fourier rời rạc (FFT 512 điểm), vậy tại sao không tìm bin có biên độ lớn nhất (argmax) ở dải tần số thấp để làm F_0?*

Cách làm này thất bại ngay từ bước kiểm tra thực nghiệm vì hai bức tường vật lý:

1. Độ phân giải tần số quá thô ($\Delta f_{\text{bin}} = 31{,}25$ Hz): Lưới FFT $N = 512$ điểm ở tần số lấy mẫu $f_s = 16.000$ Hz tạo ra bước nhảy giữa hai bin kế tiếp là:

$$\Delta f_{\text{bin}} = \frac{f_s}{N} = \frac{16.000}{512} = 31{,}25 \text{ Hz}$$

Các vạch tần số rời rạc chỉ có thể nhận giá trị: 62,5 Hz (bin 2), 93,75 Hz (bin 3), 125,0 Hz (bin 4), 156,25 Hz (bin 5)…Giả sử một giọng nam nói ở cao độ $F_0 = 105$ Hz: tần số này rơi vào giữa bin 3 và bin 4. Thuật toán chọn bin nào cũng tạo ra sai số tuyệt đối từ 11–20 Hz (sai lệch $> 15\%$). Khi đem chuỗi F_0 giật cục này tính tương quan Pearson với đường cao độ thực tế của Praat, hệ số tương quan r sẽ sụt giảm mạnh (không đạt ngưỡng tin cậy).

2. Nguy cơ bắt nhầm vào các cực cộng hưởng Formant (F_1): Dây thanh quản phát ra các xung âm thanh tam giác giàu hài âm. Khi đi qua khoang miệng, các tần số trùng với tần số cộng hưởng của khoang miệng (Formant $F_1 \approx 300$–800 Hz) sẽ được khuếch đại cực đại. Do năng lượng tại vùng cộng hưởng Formant F_1 thường lớn hơn tần số cơ bản F_0 từ 10 dB đến 25 dB, phép tìm cực đại đơn giản (argmax) trên phổ biên độ STFT **rất dễ bị đánh lừa và có nguy cơ cao bắt nhầm vào** F_1, dẫn đến lỗi nhảy bậc cao độ (octave jump).

► NHÌN THẤY VẬT LÝ

Biến đổi Fourier STFT là công cụ nhìn bao quát toàn phổ tần số, không phải cây thước kẹp đo chu kỳ dao động thanh quản. Muốn đo chu kỳ T_0, ta phải quay trở về **miền thời gian**.

2. Bản chất vật lý của Phép Tự Tương Quan (Autocorrelation)

Khi con người phát ra một nguyên âm có thanh (voiced speech), luồng khí từ phổi đẩy qua thanh môn làm hai mép dây thanh quản rung lên nhịp nhàng. Tín hiệu âm thanh $x[n]$ trong miền thời gian là một chuỗi các dạng

sóng lặp lại sau mỗi chu kỳ rung. Để phân tích định lượng, ta định nghĩa các đại lượng cơ bản:

- L: Số lượng mẫu âm thanh trong một khung phân tích. Khung dữ liệu là một mảng rời rạc gồm L phần tử, có chỉ số n chạy từ 0 đến $L-1$: $x[0], x[1], \ldots, x[L-1]$.
- T_0: Chu kỳ cơ bản của dây thanh quản, đo bằng số mẫu rời rạc ($T_0 \in \mathbb{N}^*$). Khi dây thanh rung tuần hoàn với chu kỳ T_0, dạng sóng lặp lại: $x[n] \approx x[n+T_0]$.
- τ (tau): Độ trễ (lag), là số mẫu mà ta dịch trượt bản sao tín hiệu so với bản gốc, với τ là số nguyên không âm ($\tau \geq 0$).
- F_0: Tần số cơ bản (Pitch / Cao độ), xác định qua hệ thức: $F_0 = f_s/T_0$ (với f_s là tần số lấy mẫu).

► HỎI – ĐÁP TƯ DUY: Khi đang nói, làm sao dây thanh biết trước tương lai để lặp lại mẫu $x[n+T_0]$?

Nếu đọc $x[n] \approx x[n+T_0]$ như "mẫu hiện tại bằng mẫu tương lai", trực giác sẽ vấp ngay: người nói đâu có biết trước mẫu tiếp theo để bắt buộc nó lặp lại. **Cơ thể hoàn toàn không cần biết tương lai.** Câu đúng là dạng nhân quả: $x[n] \approx x[n-T_0]$ — mẫu lúc này giống mẫu đúng một chu kỳ trước. Phép tự tương quan chỉ **đo** mức lặp lại đó; nó không bắt tín hiệu phải lặp lại. Hai lý do vật lý giải thích vì sao sự lặp lại xảy ra mà không cần tương lai:

1. **Dây thanh là một dao động cơ tự kích.** Luồng không khí từ phổi làm áp suất vùng thanh môn tụt xuống (hiệu ứng Bernoulli), kéo hai mép dây thanh hút vào nhau, cắt dòng khí rồi lại bung ra. Mỗi lần bung là một xung, và dao động này có khối lượng – độ đàn hồi riêng nên tự giữ một nhịp đều, giống dây đàn gảy xong vẫn rung theo tần số của chính nó. Xung vừa phát ra **đã định sẵn** xung kế tiếp; không có tương lai nào phải biết trước.
2. **Bộ máy phát âm thay đổi rất chậm so với nhịp rung.** Lưỡi, môi, hàm cần khoảng 50–100 ms mới đổi được một âm tiết; trong một khung 40 ms (640 mẫu) hình dạng ống cộng hưởng gần như đứng yên. Vì thế hai chu kỳ liền nhau đi qua **cùng một bộ lọc**, nên cùng bị bóp méo giống nhau và trông giống nhau.

Khi nào giả thiết này sụp đổ? (a) Âm vô thanh như /s/, /f/: không có dây thanh rung, tín hiệu gần như nhiễu ngẫu nhiên, không có chu kỳ. (b) Chuyển âm nhanh: đầu khung là một nguyên âm, cuối khung đã sang nguyên âm khác, bộ lọc không còn đứng yên nữa. Đó là lý do Praat đo cao độ theo từng khung ngắn và ghép lại bằng đường bao trơn, chứ không đo một lần cho cả câu.

Nguyên lý của phép tự tương quan rất trực quan: **Tạo ra một bản sao của khung tín hiệu, trượt bản sao đi một khoảng trễ τ mẫu, sau đó nhân từng cặp mẫu tương ứng và tính tổng dồn lại.** Biểu thức toán học này được gọi là **tổng tự tương quan** (hoặc **tổng tích các cặp mẫu**):

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x[n] \cdot x[n+\tau] \tag{1}$$

Nguồn gốc cận trên $L-1-\tau$ và số lượng cặp mẫu: Trong Công thức (1), bản gốc lấy mẫu tại chỉ số n, còn bản trượt lấy mẫu tại chỉ số $n+\tau$. Do khung dữ liệu trong bộ nhớ chỉ lưu trữ các phần tử từ chỉ số 0 đến $L-1$, mẫu của bản trượt muốn hợp lệ (không truy xuất tràn mảng) thì chỉ số phải thỏa mãn điều kiện biên:

$$n+\tau \leq L-1 \iff n \leq L-1-\tau \tag{2}$$

Do đó, chỉ số n chỉ có thể chạy từ 0 đến $L-1-\tau$. Từ điều kiện này rút ra các ý nghĩa bản chất:

1. Số lượng cặp mẫu được nhân với nhau là: $(L-1-\tau)-0+1 = \mathbf{L-\tau}$ cặp.
2. Con số "-1" xuất phát từ việc mảng đánh số từ 0 (phần tử cuối cùng là $L-1$, không phải L).
3. Đại lượng "$-\tau$" xuất hiện do khi trượt đi τ mẫu, đúng τ mẫu ở đuôi của bản trượt bị thò ra ngoài phạm vi lưu trữ của khung L mẫu (Hình 1.3.5bA).
4. Cận trên $L-1-\tau$ hoàn toàn là điều kiện biên kỹ thuật của khung dữ liệu hữu hạn, **đây KHÔNG phải là phép trừ phần đối xứng**.

Cơ chế theo dấu của tổng tự tương quan trong 3 trường hợp: Dấu và độ lớn của tổng tự tương quan $\mathcal{R}_x[\tau]$ được quyết định bởi tính đồng pha giữa bản gốc và bản trượt:

1. **Khi $\tau = 0$ (Trùng khớp):** Bản sao nằm trùng khít lên bản gốc ($x[n]$ nhân với chính nó). Từng tích cặp mẫu trở thành bình phương: $x[n] \cdot x[n] = x^2[n] \geq 0$. Mọi số hạng đều không âm, cộng dồn tạo thành:

$$\mathcal{R}_x[0] = \sum_{n=0}^{L-1} x^2[n] = E_{\text{frame}}$$

   chính là tổng năng lượng tức thời của toàn khung L mẫu. Đây là giá trị **cực đại toàn cục** của hàm tự tương quan.

2. **Khi $0 < \tau < T_0$ (Lệch pha):** Bản sao bị lệch pha so với bản gốc. Các đỉnh sóng dương $(+)$ của bản này rơi vào sườn sóng hoặc đáy sóng âm $(-)$ của bản kia. Phép nhân giữa mẫu dương và mẫu âm sinh ra tích âm: $(+) \cdot (-) < 0$. Khi cộng dồn, các tích âm và dương triệt tiêu lẫn nhau, khiến tổng tự tương quan sụt giảm rõ rệt.

3. **Khi $\tau = T_0$ (Đúng một chu kỳ cơ bản):** Sóng âm tuần hoàn lặp lại đúng một chu kỳ. Đỉnh dương của bản sao trượt đúng vào đỉnh dương của bản gốc: $(+) \cdot (+) > 0$. Đáy âm của bản sao cũng trượt đúng vào đáy âm của bản gốc: $(-) \cdot (-) > 0$. Hầu hết các cặp mẫu đều cho tích dương, khiến tổng tự tương quan vọt lên tạo thành một **đỉnh cực đại cục bộ thứ nhất** tại $\tau^* = T_0$ (Hình 1.3.5bB).

Tự tương quan đo tính tuần hoàn (lặp lại theo chu kỳ), không đo tính đối xứng; sóng tiếng nói không đối xứng nhưng tuần hoàn.

Vì sao phải bỏ qua $\tau = 0$ và thiết lập dải tìm kiếm $[\tau_{\text{min}}, \tau_{\text{max}}]$: Mặc dù $\mathcal{R}_x[0]$ luôn đạt giá trị lớn nhất, ta bắt buộc phải loại bỏ $\tau = 0$ và vùng lân cận khi tìm kiếm cao độ. Do tính liên tục của sóng âm thanh (quán tính của không khí và dây thanh), các mẫu liền kề nhau có độ tương quan rất cao, tạo thành một sườn dốc đi xuống rất dốc từ $\mathcal{R}_x[0]$. Nếu quét argmax từ $\tau = 0$, thuật toán sẽ luôn chọn nhầm $\tau = 0$ hoặc $\tau = 1$.

Dải tần số cơ bản của giọng nói con người (từ giọng nam trầm nhất đến giọng nữ cao và trẻ em) nằm trong khoảng $F_0 \in [75 \text{ Hz}, 600 \text{ Hz}]$. Tại tần số lấy mẫu chuẩn $f_s = 16.000$ Hz, ta đổi dải tần số này sang dải độ trễ mẫu τ:

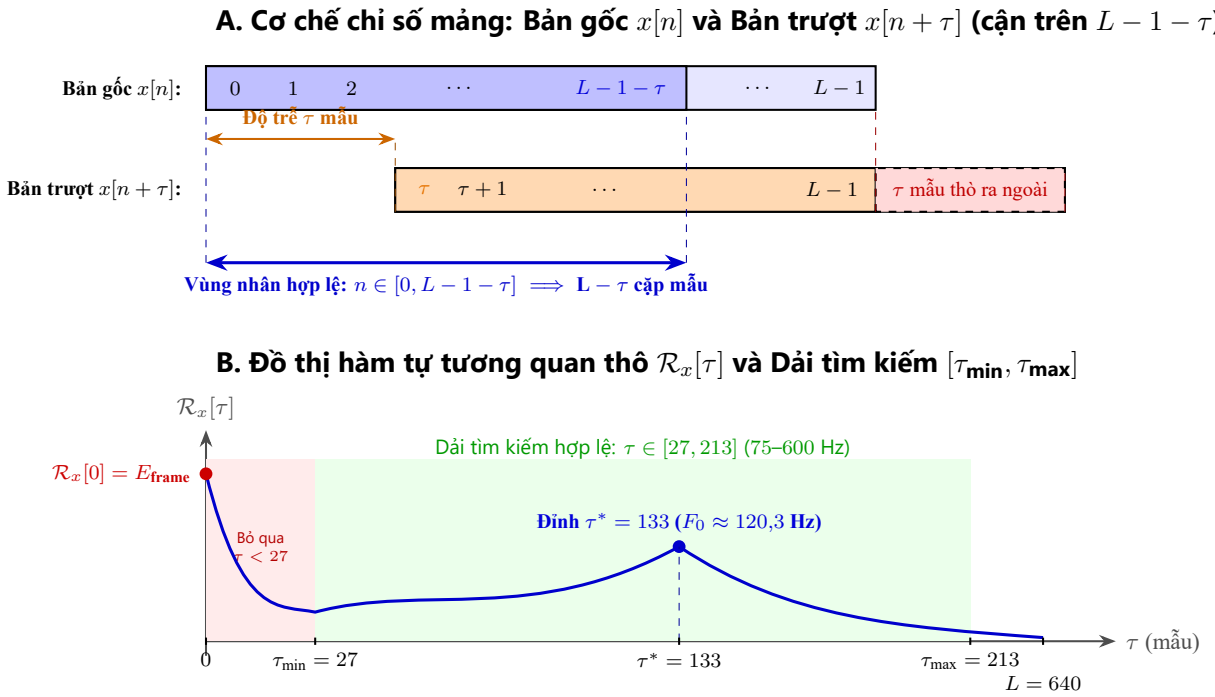
$$\tau_{\text{min}} = \left\lceil \frac{f_s}{F_{\text{ceiling}}} \right\rceil = \left\lceil \frac{16.000}{600} \right\rceil = \lceil 26{,}67 \rceil = \textbf{27} \text{ mẫu} \tag{3}$$

$$\tau_{\text{max}} = \left\lfloor \frac{f_s}{F_{\text{floor}}} \right\rfloor = \left\lfloor \frac{16.000}{75} \right\rfloor = \lfloor 213{,}33 \rfloor = \textbf{213} \text{ mẫu} \tag{4}$$

Vì vậy, không gian tìm kiếm cực đại chỉ quét trong dải hợp lệ $\tau \in [27, 213]$ gồm đúng $213 - 27 + 1 = \textbf{187}$ giá trị lag.

Ví dụ số nguyên chuỗi tính toán: Giả sử hệ thống xử lý tín hiệu tại $f_s = 16.000$ Hz:

- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 160$ mẫu $\implies F_0 = \frac{16.000}{160} = \textbf{100}$ Hz (giọng nam trầm).
- Nếu thuật toán tìm được đỉnh cực đại tại $\tau^* = 133$ mẫu $\implies F_0 = \frac{16.000}{133} \approx \textbf{120,3}$ Hz (giọng nam trung).



Hình 1.3.5b: Cơ chế chỉ số mảng giới hạn khung $(L-1-\tau)$ và đồ thị hàm tự tương quan thô $\mathcal{R}_x[\tau]$ trong dải quét $[27, 213]$ mẫu.

Đọc Hình 1.3.5b (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Bản gốc $x[n]$ (Thanh màu xanh nhạt phía trên):** Thể hiện mảng dữ liệu mẫu âm thanh của khung $L = 640$ mẫu được lưu tuần tự trong bộ nhớ, có chỉ số n chạy từ 0 đến $L-1$ (0 đến 639). Vùng tô xanh đậm hơn là đoạn chỉ số $n \in [0, L-1-\tau]$ có thể bắt cặp.
2. **Panel A — Bản trượt $x[n+\tau]$ (Thanh màu cam phía dưới):** Thể hiện bản sao tín hiệu bị dịch trượt sang phải một khoảng trễ τ mẫu. Mẫu đầu tiên của bản trượt bắt đầu tại chỉ số τ, và mẫu có chỉ số $L-1$ của bản gốc lúc này đối diện với chỉ số $L-1-\tau$ của bản trượt.
3. **Panel A — Mũi tên hai chiều màu cam (Độ trễ τ mẫu):** Đo khoảng cách dịch chuyển giữa điểm gốc 0 của bản gốc và điểm bắt đầu của bản trượt.
4. **Panel A — Khối nét đứt màu đỏ nhạt (τ mẫu thò ra ngoài):** Khi dịch trượt sang phải một khoảng τ, đúng τ mẫu ở phần đuôi của bản trượt bị tràn ra ngoài phạm vi lưu trữ của khung L mẫu. Các mẫu này không có mẫu tương ứng của bản gốc để nhân.
5. **Panel A — Mũi tên hai chiều màu xanh đậm (Vùng nhân hợp lệ):** Giới hạn trong khoảng chỉ số $n \in [0, L-1-\tau]$, tương ứng với đúng $L-\tau$ cặp mẫu có đủ cả hai thành phần $x[n]$ và $x[n+\tau]$ nằm trọn vẹn trong bộ nhớ khung. Điều này minh chứng trực quan cho điều kiện biên $n+\tau \le L-1 \iff n \le L-1-\tau$ trong Công thức (2) và giải thích vì sao cận trên của tổng tự tương quan Công thức (1) dừng lại ở $L-1-\tau$.
6. **Panel B — Chấm đỏ tại $\tau = 0$ ($\mathcal{R}_x[0] = E_{\text{frame}}$):** Biểu diễn cực đại toàn cục của hàm tự tương quan, tương ứng với năng lượng tức thời của toàn bộ khung L mẫu (khi hai bản trùng khít hoàn toàn, mọi tích cặp đều là bình phương không âm $x^2[n] \ge 0$).
7. **Panel B — Vùng tô màu hồng nhạt $\tau < 27$ (Sườn dốc quán tính âm học):** Đoạn đồ thị sụt dốc rất dốc ngay sau điểm $\tau = 0$. Sự sụt dốc này phản ánh tính tương quan rất cao giữa các mẫu âm thanh liền kề do quán tính chuyển động của dây thanh quản và cột không khí. Vùng này bị thuật toán loại bỏ triệt để để không bao giờ chọn nhầm đỉnh tại $\tau = 0$ hay $\tau = 1$.
8. **Panel B — Đáy suy giảm gần τ_{min}:** Vùng trũng của đồ thị ngay trước mốc $\tau_{\text{min}} = 27$ mẫu, nơi tương quan quán tính âm học đã tắt hẳn và bắt đầu chuyển sang vùng tìm kiếm tính tuần hoàn của giọng nói.
9. **Panel B — Đỉnh màu xanh đậm tại $\tau^* = 133$ mẫu (Cực đại cục bộ thứ nhất):** Vị trí bản trượt dịch đúng bằng chu kỳ rung cơ bản T_0 của dây thanh quản. Tại đây các đỉnh sóng dương và đáy sóng âm trùng pha nhau, làm tổng tích dồn vọt lên đỉnh cực đại cục bộ trong dải quét. Tần số cơ bản được tính chính xác bằng $F_0 = f_s/\tau^* = 16.000/133 \approx 120{,}3$ Hz (giọng nam trung).

10. **Panel B — Vùng tô màu xanh lá $\tau \in [27, 213]$ (Dải tìm kiếm hợp lệ):** Không gian quét gồm đúng 187 giá trị lag, được giới hạn bởi $\tau_{\text{min}} = 27$ mẫu (ứng với $F_{\text{ceiling}} = 600$ Hz, Công thức 3) và $\tau_{\text{max}} = 213$ mẫu (ứng với $F_{\text{floor}} = 75$ Hz, Công thức 4).

3. Cắt Khung, Chuẩn Hóa Cửa Sổ và Giải Phẫu 10 Tham Số Praat

Chuyển tiếp từ Mục 1.3: Vì sao đổi sang cửa sổ Hanning và độ dài $L = 640$ mẫu? Ở Mục 1.2 và 1.3, chuỗi tiền xử lý dùng khung $L = 400$ mẫu (25 ms) kết hợp cửa sổ Hamming. Thời lượng 25 ms là tiêu chuẩn vàng để trích xuất 80 dải Log-Mel vì đặc tính âm học của khẩu hình miệng giữ trạng thái bán dừng (quasi-stationary) trong khoảng 20–30 ms. Tuy nhiên, khi bước sang bài toán đo cao độ F_0, ta bắt buộc phải thay đổi cả độ dài khung lẫn dạng cửa sổ:

1. **Độ dài khung $L = 640$ mẫu (40 ms):** Theo quy tắc của Boersma (1993) trong Praat, khung phân tích cao độ phải chứa **tối thiểu 3 chu kỳ** của tần số thấp nhất cần đo (pitch_floor $= 75$ Hz). Chu kỳ dài nhất là $T_{\text{max}} = 16.000/75 \approx 213{,}33$ mẫu (13,33 ms). Khung phải dài ít nhất:

$$T_{\text{frame}} \geq 3 \times T_{\text{max}} = 3 \times 13{,}33 \text{ ms} = 40 \text{ ms} \implies L = 16.000 \times 0{,}040 = \mathbf{640} \text{ mẫu}$$

   Khung 400 mẫu (25 ms) của STFT chỉ chứa được 1,8 chu kỳ của giọng nam 75 Hz, hoàn toàn không đủ độ dài để bản trượt $x[n + \tau]$ bao trọn chu kỳ lặp lại thứ hai!

2. **Vì sao chọn Hanning thay vì Hamming:** Cửa sổ Hamming không triệt tiêu về 0 ở hai đầu biên ($w[0] = w[L-1] = 0{,}08$), gây ra bước nhảy gián đoạn biên khi tính tích trượt giữa hai bản tín hiệu. Ngược lại, cửa sổ Hanning vuốt mượt về đúng 0 ở cả hai đầu ($w[0] = w[L-1] = 0$), giúp hàm tự tương quan cửa sổ $\mathcal{R}_w[\tau]$ triệt tiêu êm dịu mà không sinh ra các gợn sóng biên giả tạo. Đồng thời, hàm Hanning đối xứng bậc 640 dễ dàng nén một nửa vào bảng tra ROM 320 phần tử trên FPGA, tiết kiệm tài nguyên silicon.

A. Độ dài khung $L = 640$ mẫu: Giá trị thiết kế cho dải trầm 75 Hz

Con số $L = 640$ mẫu (40 ms tại $f_s = 16.000$ Hz) là **giá trị thiết kế kỹ thuật điển hình** được quy chuẩn hóa khi ấn định giới hạn đáy cao độ pitch_floor $= 75$ Hz theo quy tắc 3 chu kỳ ở trên.

B. Cửa sổ Hanning vs. Gaussian và Hệ số chuẩn hóa $\mathcal{R}_w[\tau]$ (Boersma, 1993)

Trong thực tế, Praat ưu tiên sử dụng cửa sổ **Gaussian** vì đặc tính dập tắt búp phụ vượt trội trong miền tần số. Tuy nhiên, việc lựa chọn **cửa sổ Hanning đối xứng bậc 640** ở đây là một **lựa chọn kỹ thuật đơn giản hóa có chủ đích (deliberate engineering trade-off)**: Hanning dễ dàng sinh từ hàm cosin, đối xứng chặt chẽ, dễ nén một nửa vào ROM bảng tra, tiết kiệm tài nguyên FPGA mà vẫn đảm bảo độ mịn biên.

Tuy nhiên, việc nhân cửa sổ làm xuất hiện hiện tượng sụt dốc tự nhiên của đồ thị tự tương quan theo độ trễ τ do **hai nguyên nhân vật lý độc lập**:

1. **Nguyên nhân hình học (Số cặp mẫu giảm):** Như đã chỉ ra ở Mục 2, số cặp mẫu tham gia vào tổng tự tương quan giảm tuyến tính từ L cặp (tại $\tau = 0$) xuống còn $L - \tau$ cặp (tại độ trễ τ). Ít số hạng cộng dồn hơn khiến tổng số học đương nhiên bị suy giảm (Hình 1.3.5cA).

2. **Nguyên nhân trọng số cửa sổ (Vuốt biên):** Cửa sổ Hanning vuốt hai đầu về 0. Khi trượt bản sao đi một khoảng τ, tích của hai trọng số cửa sổ $w[n] \cdot w[n + \tau]$ suy giảm phi tuyến theo τ.

Hàm tự tương quan của chính cửa sổ phân tích:

$$\mathcal{R}_w[\tau] = \sum_{n=0}^{L-1-\tau} w[n] \cdot w[n + \tau] \tag{5}$$

gói trọn cả hai nguyên nhân nói trên (vừa phản ánh sự co hẹp của cận lấy tổng từ 0 đến $L - 1 - \tau$, vừa tích hợp sự suy giảm của tích các trọng số cửa sổ).

Trực giác trước khi viết công thức: hai quả núi Hanning trượt lên nhau.

Bước 1 — Vì sao phải vuốt khung thành "quả núi"? Nếu cắt khung L mẫu bằng một nhát kéo thẳng (cửa sổ chữ nhật), hai đầu khung kết thúc đột ngột giữa một dao động đang chạy. Chỗ gãy ấy giống như bấm tắt loa

giữa chừng: nó sinh ra các thành phần tần số cao không có trong giọng nói thật (edge discontinuity). Cửa sổ Hanning thay nhát kéo bằng một đường dốc đứng từ 0 lên đỉnh rồi xuống 0: khung có dạng một quả núi, hai chân núi chạm đất nên không còn chỗ gãy.

Bước 2 — Trượt hai quả núi lên nhau. Bản sao là quả núi thứ hai, trượt sang phải đúng τ mẫu. Khi $\tau = 0$, hai quả núi chồng khít và mép núi cao nhất vẫn nguyên. Khi τ lớn dần, đỉnh quả thứ hai rời khỏi đỉnh quả thứ nhất, và τ mẫu ở đuôi quả thứ hai thò ra ngoài cửa sổ, không còn mẫu nào của quả thứ nhất đối diện để nhân. Phần chồng nhau vừa **hẹp lại** (số cặp giảm từ L xuống $L - \tau$), vừa **thấp dần** ở hai bên vì hai chân núi chỉ còn gặp nhau ở cùng một độ cao nhỏ. Diện tích chồng nhau chính là $\mathcal{R}_w[\tau]$ — tự tương quan của **chính cửa sổ** (Hình 1.3.5cA).

Phân đôi tín hiệu thành hai lớp: "âm thanh thật" và "cái khuôn". Ở mỗi độ trễ, mỗi cặp mẫu cho ta hai thông tin tách biệt:

- Mẫu âm thanh $x[n]$ và $x[n+\tau]$ liền nhau bao nhiêu — phản ánh **tính tuần hoàn thật** của giọng.
- Hai trọng số cửa sổ $w[n]$ và $w[n+\tau]$ còn cao bao nhiêu — phản ánh **cái khuôn** mà ta đã ép lên tín hiệu.

Vì phép nhân là tuyến tính, hai lớp này nhân vào nhau:

$$\text{tích từng cặp} = \underbrace{x[n]\,x[n+\tau]}_{\text{âm thanh thật}} \cdot \underbrace{w[n]\,w[n+\tau]}_{\text{cái khuôn}} \tag{6}$$

Cộng dồn mọi cặp:

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} \big(x[n]\,x[n+\tau]\big)\big(w[n]\,w[n+\tau]\big) \tag{7}$$

và vì $w[n]\,w[n+\tau] \approx \mathcal{R}_w[\tau]/L$ theo nghĩa trung bình trên toàn khung, ta ghi gọn mức đo được của âm thanh thật là

$$r_{\text{thật}}[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_w[\tau]} \tag{8}$$

Hai đại lượng trung gian $r_x[\tau]$ và $\tilde{r}_x[\tau]$ mà tài liệu Praat hay dùng không cần đến nữa: ta chuyển thẳng từ $\mathcal{R}_x[\tau]$ sang đại lượng bù.

Bù lại cái khuôn: nhân với một hằng số lấy từ ROM. Ở $\tau = 0$ cái khuôn còn nguyên, nên mức đo được đúng bằng mức thật. Ở độ trễ lớn, cái khuôn chỉ còn một phần, khiến mức đo bị dìm xuống. Muốn lấy lại mức thật, ta chia cho phần khuôn còn lại. Phần khuôn còn lại đã được đo sẵn, chỉ phụ thuộc vào τ và cửa sổ, **không phụ thuộc vào tín hiệu**. Vì vậy ta tính trước một **hệ số phóng đại** và nạp vào ROM:

$$C[\tau] = \frac{\mathcal{R}_w[0]}{\mathcal{R}_w[\tau]} \quad (\geq 1,\ \text{tăng dần theo } \tau) \tag{9}$$

Mảng $C[\tau]$ gồm đúng **187 phần tử 16-bit** (ứng với $\tau \in [27, 213]$). Giá trị bù thực tế dùng để ra quyết định là

$$\mathcal{R}_{\text{bù}}[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau] \tag{10}$$

và chia cho năng lượng khung để so với ngưỡng:

$$\frac{\mathcal{R}_{\text{bù}}[\tau^*]}{\mathcal{R}_x[0]} \geq \theta_{\text{voice}} = 0{,}45 \iff \mathcal{R}_{\text{bù}}[\tau^*] \geq 0{,}45 \cdot \mathcal{R}_x[0] \tag{11}$$

Trên FPGA, phép so sánh này chỉ cần **một phép nhân** với hệ số lấy từ bảng tra và **một phép so sánh số nguyên**; không cần bộ chia.

▶ HỎI – ĐÁP TƯ DUY: Bảng dịch nhanh: 4 ký hiệu cốt lõi sang hình ảnh quả núi

Ký hiệu	Nó là gì trong hình ảnh quả núi	Ví dụ số tại $\tau = 133$
$\mathcal{R}_w[\tau]$	Diện tích chồng nhau của **hai vỏ núi rỗng** (chỉ có cửa sổ, không có âm thanh).	$0{,}40$ — vỏ còn 40%
$\mathcal{R}_x[\tau]$	Tự tương quan của **âm thanh thật** đã bị cái khuôn đè lên.	$0{,}90 \times 0{,}40 = 0{,}36$
$C[\tau]$	**Hệ số phóng đại** lấy từ ROM: chia cho phần vỏ còn lại.	$1{,}0/0{,}40 = 2{,}5$
$\mathcal{R}_{\text{bù}}[\tau]$	Mức thật sau khi kéo vỏ núi về đủ chiều cao.	$0{,}36 \times 2{,}5 = 0{,}90$

Đọc bảng theo một chiều: vỏ hụt → mức đo hụt theo đúng tỷ lệ đó → nhân với nghịch đảo của tỷ lệ → mức thật trở lại.

► HỎI – ĐÁP TƯ DUY: Nghịch lý nhân nghịch đảo — vì sao $\mathcal{R}_x[\tau] \cdot C[\tau]$ không bằng 1?

Câu hỏi: Hệ số bù là $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$, còn tín hiệu là $\mathcal{R}_x[\tau]$. Nhân hai thứ "nghịch đảo" với nhau, sao kết quả không ra 1?
Trả lời: Hai đại lượng này **không phải nghịch đảo của nhau**. Mẫu số của $C[\tau]$ là tự tương quan của **cửa sổ**, còn $\mathcal{R}_x[\tau]$ là tự tương quan của **tín hiệu âm thanh đã nhân cửa sổ**. Thực chất:

$$\mathcal{R}_x[\tau] \approx (\text{mức giống thật của giọng tại } \tau) \times \mathcal{R}_w[\tau] \tag{12}$$

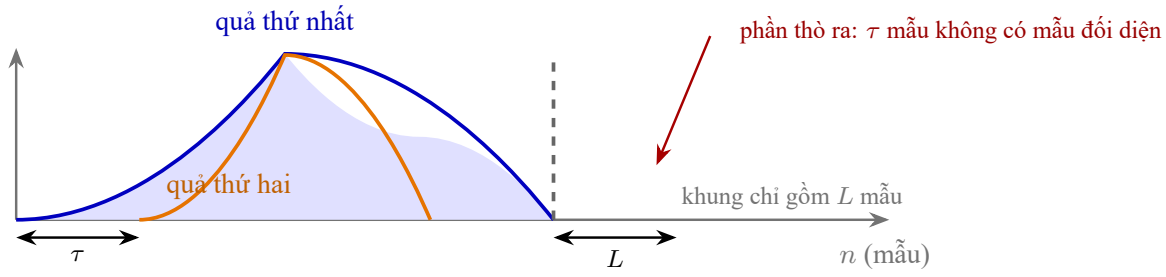
Tức là $\mathcal{R}_x[\tau]$ **đã mang sẵn** phần vỏ bên trong nó. Nhân thêm $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ là **rút** phần vỏ đó ra, còn lại mức giống thật nhân với $\mathcal{R}_w[0]$ (một hằng số). Nếu chia cho 1 thay vì nhân với $C[\tau]$, ta sẽ giữ nguyên phần vỏ bị hụt và đọc sai.
Tính từng bước với một khung giọng nam 120 **Hz (**$\tau = 133$ **mẫu), ngưỡng** $\theta_{\text{voice}} = 0{,}45$**:**

1. Vỏ núi còn: $\mathcal{R}_w[133]/\mathcal{R}_w[0] = 0{,}40$. Ở độ trễ này cái khuôn đã hụt 60%.
2. Mức đo thô: giọng thật giống nhau 0,90 ở một chu kỳ, nên $\mathcal{R}_x[133]/\mathcal{R}_x[0] = 0{,}90 \times 0{,}40 = 0{,}36$.
3. So thẳng với ngưỡng: $0{,}36 < 0{,}45 \Rightarrow$ thuật toán **nhầm là vô thanh**, mất cao độ. Đây chính là lỗi nghịch lý: giọng rất đều nhưng vẫn bị bỏ.
4. Phóng đại: $C[133] = 1{,}0/0{,}40 = 2{,}5$.
5. Sau bù: $0{,}36 \times 2{,}5 = 0{,}90$, vượt ngưỡng 0,45 rõ ràng $\Rightarrow$ bắt đúng F_0.

Liên hệ với `octave_cost` $= 0{,}01$**:** Hệ số $C[\tau]$ chỉ đo lại đúng phần vỏ đã hụt, nên **không thiên vị** đỉnh nào về mặt tần số. Việc chọn giữa đỉnh $1T_0$ và $2T_0$ lại là câu chuyện khác, được xử lý trong mục tiếp theo.

Phân biệt rõ ràng hai kiểu lỗi suy diễn cao độ:

1. **Lỗi phán nhầm Vô thanh (Voiced $\rightarrow$ Unvoiced Error):** Khi không bù suy hao cửa sổ, đỉnh tương quan ở lag lớn (ví dụ giọng nam 120 Hz tại $\tau^* = 133$) bị dìm tụt xuống mức 0,36. Vì $0{,}36 < \theta_{\text{voice}} = 0{,}45$ (ngưỡng thanh tính mặc định), thuật toán phán đoán nhầm khung này là âm vô thanh hoặc khoảng lặng và gán $F_0 = 0$, làm mất cao độ (Hình 1.3.5cC).
2. **Lỗi nhảy quãng tám (Octave Jump):** Xảy ra khi thuật toán chọn nhầm đỉnh tại chu kỳ kép $\tau = 2T_0$ (nhảy xuống quãng tám dưới $F_0/2$) hoặc đỉnh tại $\tau = T_0/2$ (nhảy lên quãng tám trên $2F_0$). Lỗi này được xử lý độc lập bởi cơ chế phạt log-lag `octave_cost` $= 0{,}01$ trong Praat, chứ không phải do việc đỉnh rơi xuống dưới ngưỡng thanh tính.



Hình 1.3.5c: Hiện tượng co hẹp vùng tích các cặp mẫu và vai trò của hệ số bù suy hao cửa sổ $C[\tau] = \mathcal{R}_w[0]/\mathcal{R}_w[\tau]$ theo Boersma (1993). Panel A: cửa sổ Hanning là một quả núi, bản sao trượt sang phải τ mẫu nên đuôi bản sao thò ra ngoài khung $L = 640$ mẫu. Panel B: mỗi cặp là hai mảnh núi kẹp vào nhau, số cặp giảm từ 640 xuống $640 - \tau$. Panel C: khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu), trục tung $R/R_x[0]$; đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) lần lượt đi từ 0,42 và 0,23 (chưa bù, đường xám nét đứt) lên khoảng 0,47 và 0,35 (đã bù $C[\tau]$, đường xanh), còn đường cam thêm `octave_cost` $= 0{,}01$. Ngưỡng thanh tính là 0,45.

Đọc Hình 1.3.5c (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Panel A — Quả núi thứ nhất (đường xanh đậm, vùng tô xanh nhạt):** Khung $L = 640$ mẫu sau khi nhân cửa sổ Hanning, có dạng một quả núi: hai chân chạm 0, đỉnh ở giữa. Cửa sổ chữ nhật sẽ để hai đầu khung gãy đột ngột; quả núi thì không.
2. **Panel A — Quả núi thứ hai (đường cam):** Bản sao của quả núi thứ nhất, trượt sang phải đúng τ mẫu. Mũi tên hai chiều ở đáy đo đúng khoảng trượt τ này.
3. **Panel A — "Phần thò ra":** τ mẫu ở đuôi quả thứ hai nằm ngoài cửa sổ, không có mẫu nào của quả thứ nhất đối diện để nhân. Mũi tên đỏ chỉ vào chỗ đó. Cùng lúc, phần chồng nhau hẹp lại còn $L - \tau$ cặp và hai chân núi chỉ còn gặp nhau ở độ cao nhỏ, nên phần chồng vừa hẹp vừa thấp. Diện tích chồng nhau chính là $\mathcal{R}_w[\tau]$.
4. **Panel B — Bốn cột mẫu:** Mỗi cột là một độ trễ $\tau = 0, 80, 160$ và một cột nữa xa hơn. Cột xanh là mảnh núi "bên trái", cột cam là mảnh "bên phải"; mũi tên "kẹp" chỉ cặp mảnh đang dán vào nhau. Từ trên xuống dưới, mảnh cam thấp dần, cho thấy mỗi trọng số tự rơi về 0.

5. **Panel B — Số cặp giảm:** Nhãn dưới cột ghi $\tau = 0$: 640 cặp; $\tau = 80$: 560 cặp; $\tau = 160$: 480 cặp. Con số $L - \tau$ này là nguyên nhân hình học đầu tiên khiến tổng tương quan thô sụt theo τ.
6. **Panel C — Trục tung $R/R_x[0]$ và ngưỡng $0{,}45$:** Khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu). Đường đỏ nét đứt ngang là ngưỡng thanh tính; trục hoành đánh dấu bốn mốc 27, 80, 160, 213.
7. **Panel C — Đường xám nét đứt (chưa bù):** Hai đỉnh $1T_0$ ($\tau = 80$) và $2T_0$ ($\tau = 160$) chỉ đạt $0{,}42$ và $0{,}23$; cả hai đều **dưới** ngưỡng $0{,}45$, nên khung bị gán vô thanh.
8. **Panel C — Đường xanh đậm (đã bù $C[\tau]$):** Nhân với $C[80] = 1{,}11$ và $C[160] = 1{,}52$, hai đỉnh lên khoảng $0{,}47$ và $0{,}35$; cả hai đều **vượt** ngưỡng. Mũi tên đỏ giữa chấm xám và chấm xanh cho thấy đúng mức nâng bù tại mỗi đỉnh.
9. **Panel C — Đường cam (thêm `octave_cost` $= 0{,}01$):** Đường cam gần như trùng đường xanh, chỉ hơi thấp hơn ở đỉnh. Mức phạt nhỏ này đủ để đỉnh $1T_0$ đứng trên đỉnh $2T_0$ khi hai đỉnh xấp xỉ nhau, nhưng không lật ngược một đỉnh vốn đã thua rõ.
10. **Panel C — Dòng chữ xanh đậm ở đáy:** Tóm một câu: đỉnh $1T_0$ (80 mẫu) thắng đỉnh $2T_0$ (160 mẫu) nhờ `octave_cost` $= 0{,}01$.

B'. Hai đỉnh $1T_0$ và $2T_0$: vì sao bù hệ số lại đẩy đỉnh xa lên, và `octave_cost` dọn nốt tàn dư

Một giọng có chu kỳ T_0 sẽ lặp lại không chỉ sau một chu kỳ mà còn sau $2T_0$, $3T_0$… nên hàm tự tương quan có **nhiều đỉnh**. Chọn nhầm đỉnh $2T_0$ là đọc cao độ thành $F_0/2$: giọng 200 Hz bị ghi thành 100 Hz, tức **rơi đúng một quãng tám** (octave error). Đây là lỗi nguy hiểm nhất của bộ bắt cao độ tự tương quan.

Vì sao trước khi bù, đỉnh $2T_0$ thấp hơn. Ở $\tau = 2T_0$, vỏ núi đã hụt nhiều hơn ở $\tau = T_0$, nên mức đo thô bị dìm nhiều hơn, dù "độ giống thật" của giọng ở hai độ trễ này gần như bằng nhau.

Vì sao sau khi bù, hệ số lại đẩy đỉnh xa lên mạnh hơn. $C[\tau]$ tăng dần theo τ; đỉnh ở $2T_0$ nằm xa hơn nên có $C[2T_0] > C[T_0]$. Hệ số bù vì vậy **nâng đỉnh $2T_0$ nhiều hơn** đỉnh T_0 và có thể kéo hai đỉnh về ngang nhau, tạo nguy cơ lỗi quãng tám thật sự.

Praat chặn nguy cơ đó bằng `octave_cost` $= 0{,}01$. Với mỗi bậc độ trễ tăng gấp đôi, Praat **trừ đi** $0{,}01$ khỏi điểm của đỉnh. Đỉnh $2T_0$ bị trừ một lần $0{,}01$ so với đỉnh T_0, nên đỉnh $1T_0$ luôn **thắng an toàn** khi hai đỉnh xấp xỉ nhau. Vì mức phạt chỉ $0{,}01$ trên thang $[0, 1]$, nó đủ để phá thế ngang bằng nhưng **không đủ để lật ngược** một đỉnh $1T_0$ đã thực sự yếu hơn nhiều — nhờ vậy giọng trầm (đỉnh $1T_0$ nằm ở lag lớn) vẫn sống sót sau khi đã được $C[\tau]$ bù.

Ví dụ số, khung $F_0 = 200$ Hz ($T_0 = 80$ mẫu). Ở khung này cả $1T_0 = 80$ lẫn $2T_0 = 160$ đều nằm trong dải quét $[27, 213]$:

1. Chưa bù: đỉnh $1T_0$ đạt $0{,}42$, đỉnh $2T_0$ đạt $0{,}23$ — cả hai đều dưới ngưỡng $0{,}45$, khung bị gán vô thanh.
2. Nhân hệ số bù: $C[80] = 1{,}11$ và $C[160] = 1{,}52$. Đỉnh $1T_0$ lên $0{,}42 \times 1{,}11 \approx 0{,}47$; đỉnh $2T_0$ lên $0{,}23 \times 1{,}52 \approx 0{,}35$.
3. Thêm `octave_cost`: $0{,}47 \times 2^{-0{,}01} \approx 0{,}47$ và $0{,}35 \times 2^{-0{,}01} \approx 0{,}35$. Cả hai đều tụt một chút, nhưng **tỷ lệ giữa hai đỉnh** chính là thứ quyết định, và đỉnh $1T_0$ đang cao hơn hẳn — lỗi quãng tám bị chặn.
4. Trường hợp nguy hiểm là khi hai đỉnh gần ngang nhau: nếu sau bù $2T_0$ đạt $0{,}90$ và $1T_0$ đạt $0{,}62$, thì phạt $0{,}01$ **không đủ** để lật; vì vậy Praat còn thêm cơ chế Viterbi trên cả chuỗi khung để giữ quỹ đạo cao độ liên tục.

Hình 1.3.5c (Panel C) vẽ đúng ba đường của ví dụ trên: đường xám nét đứt (chưa bù) nằm dưới ngưỡng, đường xanh đậm (đã bù $C[\tau]$) vượt ngưỡng, và đường cam (thêm `octave_cost`) chỉ nhích nhẹ nhưng đủ để đỉnh $1T_0$ đứng trên đỉnh $2T_0$.

C. Giải phẫu 10 tham số chuẩn của Praat dưới góc nhìn mã nguồn (`Sound_to_Pitch.cpp`)

Mã nguồn mở C++ của Praat do Paul Boersma xây dựng chứa đựng triết lý thiết kế chuẩn mực đã qua hơn 30 năm kiểm chứng. Dưới đây là giải phẫu chi tiết 10 tham số đầu vào của hàm `Sound_to_Pitch_ac`:

1. **`time_step = 0,01 s` (10 ms $= 160$ mẫu):** Bước nhảy trượt khung (Δt_{hop}). Trong Praat, nếu người dùng truyền `time_step = 0.0` (Auto), mã nguồn tự động gán $\Delta t_{\text{hop}} = \frac{0{,}75}{\text{pitch_floor}} = \frac{0{,}75}{75} = 0{,}01$ s (ứng

với 4 khung phân tích gối lên nhau trên mỗi độ dài 3 chu kỳ). Ta khóa cứng tường minh $0{,}01$ s để giữ tốc độ 100 khung/s đồng bộ với luồng Log-Mel.

2. **`pitch_floor = 75 Hz`:** Giới hạn đáy cao độ. Quyết định hai đại lượng phần cứng cốt lõi: (a) Độ trễ quét tối đa $\tau_{\max} = \lfloor f_s/75 \rfloor = 213$ mẫu; (b) Độ dài cửa sổ phân tích tối thiểu $L = \frac{3}{\text{pitch_floor}} \times f_s = 0{,}04 \text{ s} \times 16.000 = 640$ mẫu.
3. **`pitch_ceiling = 600 Hz`:** Giới hạn trần cao độ (bao phủ trọn vẹn cả giọng nữ cao và trẻ em). Quyết định độ trễ quét tối thiểu $\tau_{\min} = \lceil f_s/600 \rceil = 27$ mẫu. Mọi đỉnh tương quan ở $\tau < 27$ đều bị loại bỏ ngay lập tức như nhiễu tần số cao hoặc cực Formant.
4. **`very_accurate = off` (False):** Cờ boolean quyết định loại cửa sổ và độ dài phân tích:
   - Khi `off` (Mặc định): Praat dùng cửa sổ **Hanning** dài **3 chu kỳ** ($3/\text{pitch_floor} = 40$ ms $= 640$ mẫu).
   - Khi `on`: Praat chuyển sang cửa sổ **Gaussian** dài **6 chu kỳ** (80 ms $= 1.280$ mẫu) để dìm búp phụ xuống -120 dB. Chọn `very_accurate = off` trên FPGA là hoàn toàn phù hợp với thiết kế cửa sổ Hanning 640 mẫu, giúp giảm một nửa dung lượng BRAM và số phép tính MAC!
5. **`max_number_of_candidates = 15`:** Trong mỗi khung 10 ms, Praat không chỉ chọn 1 đỉnh cao nhất mà trích xuất tối đa **14 đỉnh cực đại cục bộ** của $r_x[\tau]$ cộng với **1 ứng viên Vô thanh (Unvoiced candidate)** (tổng $N_{\text{cand}} = 15$) để chuyển sang mạng lưới quy hoạch động Viterbi. *(Trên FPGA streaming: rút gọn về max_candidates $= 1$, tức dùng bộ so sánh lấy đỉnh cao nhất duy nhất cục bộ)*.
6. **`silence_threshold = 0,03`:** Ngưỡng biên độ khoảng lặng so với biên độ đỉnh toàn cục của file ($I_{\text{local}}/I_{\text{max_global}}$). Nếu một khung có năng lượng quá nhỏ ($< 3\%$ đỉnh toàn cục), khung đó bị nghi ngờ là khoảng lặng hoặc tiếng ồn nền phòng thu.
7. **`voicing_threshold = 0,45`:** Ngưỡng độ mạnh chu kỳ (Harmonicity). Trong code Boersma (Công thức 23), ứng viên Vô thanh (Unvoiced) được gán một điểm số cạnh tranh:

$$R_{\text{unvoiced}} = \text{voicing_threshold} + \max\left(0,\ 2 - \frac{I_{\text{local}}/I_{\text{max_global}}}{\text{silence_threshold}/(1 + \text{voicing_threshold})}\right)$$

   Khi âm thanh đủ to, số hạng thứ hai bằng $0 \implies R_{\text{unvoiced}} = 0{,}45$. Một đỉnh hữu thanh muốn chiến thắng phải có tương quan chuẩn hóa $\geq 0{,}45$. Khi âm thanh rơi vào vùng khoảng lặng, R_{unvoiced} vọt lên $2{,}45$, ép khung đó nhận nhãn Unvoiced ($F_0 = 0$) bất chấp nhiễu nền tuần hoàn giả tạo!
8. **`octave_cost = 0,01` (trên mỗi quãng tám):** Hệ số thiên vị tần số cao nội khung (Local pitch-lag bias). Vì tín hiệu tuần hoàn ở T_0 cũng tuần hoàn ở $2T_0$ (tần số nửa $F_0/2$ — lỗi nhảy xuống quãng tám dưới), Praat trừ một khoản phạt tỉ lệ với logarit độ trễ (Công thức 24 Boersma):

$$R_{\text{voiced}}[\tau] = r_x[\tau] - \text{octave_cost} \times \log_2\left(\frac{\text{pitch_ceiling}}{f_s/\tau}\right)$$

   Ở lag càng lớn (τ lớn, tức F_0 càng thấp), lượng trừ càng lớn, giúp đỉnh cơ bản thật $\tau = T_0$ thắng đỉnh bội số $\tau = 2T_0$. Trên FPGA, lượng phạt này được cộng trừ sẵn vào bảng tra ROM $C[\tau]$ mà không tốn mạch tính $\log_2$!
9. **`octave_jump_cost = 0,35`:** Trọng số phạt trong thuật toán Viterbi khi đường cao độ giữa hai khung liên tiếp bị nhảy vọt tần số:

$$\text{Cost}_{\text{jump}}(F_{0,t-1} \to F_{0,t}) = \text{octave_jump_cost} \times \left|\log_2\left(\frac{F_{0,t}}{F_{0,t-1}}\right)\right| \times \left(\frac{0{,}01}{\Delta t_{\text{hop}}}\right)$$

   Nhờ mức phạt $0{,}35$ cho mỗi quãng tám, Viterbi ưu tiên chọn chuỗi ứng viên tạo ra đường cong cao độ mượt mà.
10. **`voiced_unvoiced_cost = 0,14`:** Trọng số phạt trong Viterbi khi trạng thái chuyển từ Hữu thanh sang Vô thanh (Voiced $\leftrightarrow$ Unvoiced), giúp ngăn hiện tượng đường Pitch bị đứt gãy chớp nháy ở ranh giới âm yếu.

C'. Vì sao mỗi con số tham số lại là con số đó: cơ sở âm học của Tham số 6–10

Mục C vừa rồi liệt kê mười tham số và giá trị mặc định của chúng. Nhưng một kỹ sư chuẩn bị viết RTL cần biết nhiều hơn "Praat đặt bằng 0,45": cần biết **vì sao 0,45 chứ không phải 0,30 hay 0,70**, và cần biết điều gì sẽ hỏng nếu tự ý đổi. Năm tham số dưới đây đều là những ngưỡng vật lý đã được quy đổi sang thang điểm của thuật toán, nên mỗi con số đều có thể kiểm chứng lại bằng một phép tính.

Tham số 6 — `silence_threshold = 0,03`: ngưỡng để nhiễu nền không tự khuếch đại chính nó. Khi một khung là khoảng lặng, năng lượng $\mathcal{R}_x[0]$ rất nhỏ. Nhưng phép chuẩn hóa lại chia cho chính năng lượng nhỏ đó:

$$r_x[\tau] = \frac{\mathcal{R}_x[\tau]}{\mathcal{R}_x[0]}.$$

Một tạp âm nền tuần hoàn rất yếu — tiếng quạt máy tính, hay hài bậc thấp của nhiễu nguồn $50/60$ Hz — vẫn có thể đạt tỷ số $r_x[\tau] > 0{,}45$ sau khi chia, *không phải vì nó mạnh, mà vì mẫu số đã bé tí*. Nếu không có ngưỡng tuyệt đối, bộ dò sẽ báo cao độ giả trong đúng những khoảng lặng mà nó phải im.

Ngưỡng $0{,}03$ chính là một mức SNR quy ra decibel:

$$20 \log_{10}(0{,}03) \approx -30{,}46 \text{ dB}.$$

Con số này không tùy tiện: nó trùng với khoảng cách điển hình giữa giọng nói bình thường (khoảng 65 dB SPL) và nền phòng yên tĩnh (khoảng 35 dB SPL), tức đúng 30 dB. Đó cũng là lý do `top_db = 30` của Librosa ở Phụ lục cho ra cùng một ngưỡng biên độ: $10^{-30/20} \approx 0{,}0316$. Hai công cụ khác nhau, một ngưỡng vật lý.

Tham số 7 — `voicing_threshold = 0,45`: mốc cân bằng giữa tuần hoàn và nhiễu. Khi phát âm /s/ hay /f/, biên độ rất lớn — vượt xa ngưỡng lặng — nhưng tín hiệu là nhiễu khí hỗn loạn, không có chu kỳ thanh quản. Ngưỡng lặng không bắt được trường hợp này; cần một ngưỡng riêng cho *tính tuần hoàn*.

Thước đo chuẩn là **Tỷ số Hài trên Nhiễu** (Harmonics-to-Noise Ratio, HNR — tỷ lệ năng lượng phần tuần hoàn trên phần nhiễu). Boersma (1993) nối HNR với hệ số tự tương quan chuẩn hóa r bằng

$$\text{HNR (dB)} = 10 \log_{10}\left(\frac{r}{1-r}\right).$$

Điểm cân bằng vật lý — năng lượng tuần hoàn đúng bằng năng lượng nhiễu — là HNR $= 0$ dB, và phương trình trên cho $r = 0{,}50$ tại đúng điểm đó. Praat chọn $r = 0{,}45$, tương ứng

$$10 \log_{10}\left(\frac{0{,}45}{0{,}55}\right) \approx -0{,}87 \text{ dB},$$

tức chấp nhận một khung mà nhiễu *nhiều hơn* phần tuần hoàn khoảng $0{,}9$ dB. Đó là một dung sai nhỏ nhưng có chủ đích: nó giữ được các nguyên âm hơi khàn ở cuối câu, vốn tụt xuống dưới mốc cân bằng, mà không mở cửa cho phụ âm xát.

Tham số 8 — `octave_cost = 0,01`: mức phạt nhỏ nhất còn đủ để phá thế hòa. Đây là tham số mà trực giác hay đánh giá thấp nhất, và cũng là tham số cuốn sách này đã đo lại. Tín hiệu tuần hoàn ở T_0 cũng tuần hoàn ở $2T_0$, nên sau khi bù cửa sổ, hai đỉnh có thể **hòa nhau**. Phép đo trong `lab_1_3_5.py` cho thấy với một âm 200 Hz, đỉnh $\tau = 80$ và đỉnh $\tau = 160$ đều đạt đúng $1{,}000$ sau khi bù — một bộ `argmax` trần trụi sẽ chọn nhầm và báo 100 Hz cho một âm 200 Hz.

Quét toàn dải 75–600 Hz trên lưới 1 Hz (526 tần số) cho kết quả:

`octave_cost`	Số tần số bị đọc sai quãng tám	Ghi chú
0	$273/526$	quá nửa dải sai: hai đỉnh hòa nhau
$0{,}005$	$20/526$	còn sai ở vùng cao
$0{,}009$	$0/526$	sạch
0,01	**0/526**	**mặc định Praat**
$0{,}02$	$0/526$	sạch, dư an toàn

Ngưỡng sạch nằm ở khoảng $0{,}0089$. Giá trị $0{,}01$ của Praat vượt ngưỡng đó khoảng 12%: đủ chặt để không có tần số nào trong dải bị đọc nhầm, nhưng vẫn đủ nhỏ để không dìm một đỉnh thật. Con số 1% này cũng cùng bậc với độ bất ổn chu kỳ tự nhiên của dây thanh (jitter điển hình cỡ $0{,}5$–1% ở giọng bình thường), nên nó phạt vừa đúng mức dao động mà cơ thể vẫn tạo ra một cách tự nhiên.

Sửa một công thức: dấu và mốc quy chiếu của `octave_cost`. Đoạn văn ở Mục C phát biểu lượng phạt là octave_cost $\times \log_2(\text{pitch_floor} \cdot \tau / f_s)$. Đối chiếu mã nguồn `fon/Pitch.cpp` thì công thức đúng là

$$\text{score} = \text{strength} - \text{octave_cost} \times \log_2\left(\frac{\text{ceiling}}{F_0}\right),$$

tức mốc quy chiếu là **trần `ceiling`** chứ không phải `floor`, và đối số là nghịch đảo. Hai dạng này khác nhau cả về dấu lẫn điểm không: dạng đúng cho lượng phạt bằng 0 tại trần 600 Hz và lớn nhất tại đáy 75 Hz (đúng như chú thích "High frequencies are to be favoured" trong mã nguồn), còn dạng trong sách cũ cho lượng phạt *âm* ở vùng cao, nghĩa là thưởng cho tần số cao thay vì phạt tần số thấp. Trên FPGA, hệ quả thực hành không đổi — cả hai dạng đều gộp được vào một bảng tra — nhưng con số trong bảng tra phải tính theo dạng đúng.

Tham số 9 và 10 — `octave_jump_cost = 0,35` và `voiced_unvoiced_cost = 0,14`: quán tính cơ học của thanh quản. Hai tham số này chỉ tồn tại trong thuật toán Viterbi, vì chúng phạt *bước chuyển* giữa hai khung liên tiếp chứ không phạt một khung đơn lẻ. Cơ sở của chúng là một giới hạn sinh lý: cao độ do độ căng của cơ nhẫn giáp (cricothyroid) và cơ giáp phễu (thyroarytenoid) trong thanh quản quyết định. Nghiên cứu cơ sinh học phát âm kinh điển của Sundberg (1979) chỉ ra rằng thanh quản người cần ít nhất 80–100 ms để co duỗi thay đổi độ căng qua một quãng tám (F_0 gấp đôi). Trong một bước nhảy trượt chỉ vỏn vẹn 10 ms (Δt_{hop}), việc cao độ đột ngột nhảy vọt một quãng tám là điều bất khả thi về mặt sinh học. Biến động thực tế giữa hai khung 10 ms hiếm khi vượt quá 1–2 nửa cung (semitones).

Nhìn vào con số để thấy mức phạt được cân như thế nào. Với $\Delta t_{\text{hop}} = 10$ ms:

- Nhảy đúng một quãng tám bị phạt $0{,}35 \times \log_2(2) = 0{,}35$.
- Nhảy một nửa cung (semitone, tỷ số $2^{1/12}$) chỉ bị phạt $0{,}35 \times \frac{1}{12} \approx 0{,}029$.

Tỷ lệ này — hơn mười lần — là toàn bộ nội dung của tham số: Viterbi được phép uốn éo thoải mái trong phạm vi nửa cung, nhưng phải trả giá rất đắt nếu muốn nhảy quãng tám giữa hai khung liền nhau. Mức $0{,}14$ cho chuyển hữu thanh/vô thanh cũng theo logic đó, nhưng nhẹ hơn, vì chuyển trạng thái thanh quản bật/tắt nhanh hơn nhiều so với đổi cao độ một quãng tám.

Tham số Praat	Ký hiệu	Giá trị chuẩn	Ý nghĩa thuật toán & Mã nguồn (Boersma 1993)	Hiện thực trên FPGA DE10-Standard
`time_step`	Δt_{hop}	$0{,}01$ s	Bước nhảy trượt khung (160 mẫu tại 16 kHz)	Khóa cứng 10 ms (100 khung/s) đồng bộ luồng Mel
`pitch_floor`	F_{floor}	75 Hz	Giới hạn đáy cao độ $\implies \tau_{\max} = 213$ mẫu, $L = 640$ mẫu	Cửa sổ 640 mẫu lưu trong 1 khối M10K BRAM
`pitch_ceiling`	F_{ceiling}	600 Hz	Giới hạn trần cao độ $\implies \tau_{\min} = 27$ mẫu (187 lag)	Quét 187 độ trễ $\tau \in [27, 213]$, bỏ qua $\tau < 27$
`very_accurate`	—	`off`	Cửa sổ Hanning 3 chu kỳ (640 mẫu) vs Gaussian 6 chu kỳ	Dùng Hanning 640 mẫu; tiết kiệm 50% BRAM và MAC
`max_candidates`	N_{cand}	15	14 đỉnh cực đại cục bộ + 1 ứng viên Vô thanh cho Viterbi	Rút gọn về 1 ứng viên (bộ so sánh argmax cực đại)
`silence_threshold`	θ_{silence}	$0{,}03$	Ngưỡng biên độ khoảng lặng so với đỉnh file (3%)	Ngưỡng năng lượng tĩnh $E_{\text{frame}} > E_{\text{silence}}$ trước khi đo
`voicing_threshold`	θ_{voice}	$0{,}45$	Ngưỡng tỷ số tự tương quan chuẩn hóa nhận diện hữu thanh	Điều kiện $\mathcal{R}_{\text{comp}}[\tau^*] \geq 0{,}45 \cdot \mathcal{R}_x[0]$
`octave_cost`	—	$0{,}01$	Phạt thiên vị log-lag nội khung để tránh nhảy xuống $F_0/2$	Gộp sẵn lượng bù vào hệ số bảng tra ROM $C[\tau]$
`octave_jump_cost`	—	$0{,}35$	Phạt nhảy quãng tám giữa hai khung liên tiếp trong Viterbi	Bỏ qua trên streaming pipeline; lọc trung vị nhẹ ở sau
`voiced_unvoiced_cost`	—	$0{,}14$	Phạt chuyển đổi Voiced $\leftrightarrow$ Unvoiced giữa hai khung	Bỏ qua; xử lý trực tiếp trên từng khung độc lập

Bảng 1.3.5a: Giải phẫu 10 tham số chuẩn mực đo cao độ của Praat và ánh xạ vi kiến trúc lên FPGA DE10-Standard.

Thẻ công thức

Công thức.

$$\mathcal{R}_x[\tau] = \sum_{n=0}^{L-1-\tau} x_{\text{win}}[n] \cdot x_{\text{win}}[n+\tau], \qquad \mathcal{R}_{\text{comp}}[\tau] = \mathcal{R}_x[\tau] \cdot C[\tau]$$

$$F_0 = \begin{cases} \dfrac{f_s}{\tau^*}, & \text{nếu } (E_{\text{frame}} > E_{\text{silence}}) \text{ và } \dfrac{\mathcal{R}_{\text{comp}}[\tau^*]}{\mathcal{R}_x[0]} \geq \theta_{\text{voice}} \\ 0 \text{ (Unvoiced)}, & \text{ngược lại} \end{cases}$$

$$\text{với } \tau^* = \arg\max_{\tau \in [27,213]} \mathcal{R}_{\text{comp}}[\tau].$$

Các biến.

- τ — độ trễ tương quan, là số nguyên quét trong dải $[27, 213]$ mẫu (187 độ trễ).
- $x_{\text{win}}[n] = x[n] \cdot w[n]$ — mẫu âm thanh sau khi nhân cửa sổ Hanning 640 điểm.
- $C[\tau] = 1/\mathcal{R}_w[\tau]$ — hệ số bù độ dốc cửa sổ lưu trong bảng tra ROM 187 phần tử.
- $\mathcal{R}_x[0]$ — năng lượng tại lag 0, dùng làm mốc so sánh thanh tính (Voicing Threshold).
- $\theta_{\text{voice}} = 0{,}45$ — ngưỡng quyết định có thanh; nếu không vượt qua, gán $F_0 = 0$.

Ý nghĩa. Tách bạch bài toán cao độ thành hai pha: (1) Kiểm tra tính chu kỳ và khoảng lặng; (2) Nếu có chu kỳ, trích xuất độ trễ lag chuẩn xác theo đơn vị mẫu rời rạc.
Giá trên silicon. Hoàn toàn chỉ dùng các bộ nhân-tích lũy (MAC) và bộ so sánh thanh ghi. Không cần bộ chia số học hay biến đổi ma trận.
Điều công thức không nói. Ở tần số cao, khoảng cách giữa hai giá trị lag nguyên kế tiếp thô hơn ở tần số thấp do quan hệ nghịch đảo $F_0 = f_s/\tau$.

4. Hành trình từ Laptop lên FPGA: Tối giản hóa First-Principles

▶ CỔNG KIỂM CHỨNG PHẦN MỀM (SOFTWARE VERIFICATION GATE)

Trước khi đưa thiết kế lên FPGA, bắt buộc phải dựng một mô hình Python tối giản (Bit-accurate Fixed-Point Model) mô phỏng đúng khung 640 mẫu, cửa sổ Hanning và 187 lag số nguyên. Đem mô hình này chạy trên tập dữ liệu âm thanh đã lọc tiền xử lý (HPF 60 Hz ở Mục 1.1.5) để đối sánh trực tiếp với đường Pitch chuẩn của Praat (đo sai số lệch chuẩn Hz và hệ số tương quan Pearson r). Chỉ khi phiên bản Python tối giản này chứng minh được độ suy giảm chất lượng nằm trong ngưỡng chấp nhận được của bài toán nhận diện thanh điệu/ngữ điệu, các con số hạch toán phần cứng bên dưới mới có giá trị thực tiễn.

A. Bản chất *bit-accurate fixed-point*: từ `float64` trên laptop đến thanh ghi số nguyên trên FPGA

Ba tầng số, và tầng nào thực sự đến được silicon. Một bản mô phỏng Python chạy được chưa chứng minh được gì về phần cứng. Trước khi viết một dòng Verilog, cần trả lời câu hỏi hẹp hơn nhiều: *phép tính nào trong thuật toán này có thể biểu diễn bằng số nguyên, và khi biểu diễn như vậy thì kết quả có còn đúng không?*

Ba cách biểu diễn số cùng xuất hiện trong chương này:

1. **`float64` (dấu phẩy động, 64-bit):** dùng trên laptop. Mỗi số gồm dấu, phần mũ và phần định trị, nên biểu diễn được dải động rất rộng ($\approx 10^{\pm 308}$). Cái giá là mỗi phép cộng/nhân phải xử lý phần mũ, và các hàm siêu việt như cos, $\log_2$ được cài bằng *xấp xỉ đa thức* khác nhau trên từng kiến trúc CPU.
2. **Q1.15 (fixed-point, 16-bit):** một số nguyên 16-bit có dấu, trong đó 1 bit dấu, 15 bit lẻ. Giá trị thực được hiểu là $x_{\text{int}}/2^{15}$, nên dải biểu diễn là $[-1, +0{,}99997]$. Đây là định dạng tự nhiên cho mẫu âm thanh 16-bit và cho hệ số cửa sổ (luôn nằm trong $[0, 1]$).
3. **Q2.30 và thanh ghi tích lũy 42-bit:** tích của hai số Q1.15 là một số 32-bit với 30 bit lẻ (Q2.30). Cộng dồn 640 tích như vậy cần thêm $\lceil \log_2 640 \rceil = 10$ bit, nên thanh ghi tích lũy phải rộng ít nhất $32 + 10 = 42$ bit.

Vì sao `float64` không bảo đảm giống nhau giữa AMD64 và ARM64, còn số nguyên thì có. Đây là điểm mấu chốt, và nó không phải chuyện "sai số làm tròn nhỏ".

Chuẩn IEEE-754 quy định chính xác kết quả của `+`, `-`, `*`, `/` trên `float64`: bốn phép này *được làm tròn đúng* (correctly rounded) và cho cùng bit trên mọi máy tuân thủ chuẩn. Nhưng `cos`, `log2`, `sin` **không** nằm trong nhóm đó. Chuẩn không quy định kết quả cuối cùng của chúng, chỉ đòi hỏi độ chính xác trong một khoảng ULP (Unit in the Last Place — đơn vị ở vị trí cuối, tức bước nhảy nhỏ nhất giữa hai số biểu diễn được). Vì vậy glibc trên AMD64 và thư viện toán học của Apple trên ARM64 có thể trả về hai kết quả lệch nhau một vài ULP cho cùng một đầu vào. Với một phép $\log_2$, sai lệch đó vô hại. Nhưng khi $\log_2$ nằm trong một bộ so sánh "đỉnh nào cao hơn", hai ULP đủ để lật ngôi một đỉnh — và lúc đó hai máy cho hai chuỗi F_0 khác nhau.

Số nguyên bù hai không có vấn đề này. Với `int16`, `int32`, `int64` và bốn phép `+`, `-`, `*`, `>>`:

- Không có khái niệm làm tròn: kết quả là số nguyên chính xác, trừ khi tràn.
- Tràn trong bù hai là hành vi *xác định*: nó quay vòng theo modulo 2^N, giống hệt nhau trên mọi kiến trúc.
- Dịch phải số học `>>` trên số có dấu cũng xác định (mở rộng bit dấu).

Nghĩa là: **cùng một chuỗi lệnh số nguyên, cùng một đầu vào, cho đúng cùng một dãy bit trên AMD64, trên ARM64, và trên mạch RTL tổng hợp từ cùng biểu thức đó**. Đó chính là định nghĩa của *bit-accurate*: không phải "gần đúng", mà "giống đến từng bit". Đây là lý do mọi so sánh trong phần còn lại của mục này — ngưỡng thanh tính, ngưỡng lặng, chọn đỉnh — đều được viết lại thành phép nhân và so sánh số nguyên, không một phép chia nào.

▶ NHÌN THẤY VẬT LÝ

Hệ quả thực hành: nếu bản Python dùng `float64` mà bản FPGA dùng số nguyên, hai bên *sẽ* lệch nhau, và mọi khác biệt đo được sẽ lẫn lộn giữa "do lượng tử hóa" và "do thuật toán". Viết bản Python bằng đúng số nguyên ngay từ đầu tách đôi hai nguyên nhân đó: khi bản Python và bản RTL khác nhau, đó là lỗi thiết kế chứ không phải sai số.

Mã nguồn: thuật toán từ con số 0, không gọi thư viện hộp đen. Đoạn mã dưới đây là phần lõi của `chapter01/lab_1_3_` chạy được và có kiểm thử. Nó chỉ dùng `numpy` để chứa mảng; mọi phép tính là số nguyên.

```
Q15, Q13 = 1 << 15, 1 << 13          # 1.0 in Q1.15 and Q3.13
TAU_MIN, TAU_MAX = 27, 213           # lag range for 600 Hz .. 75 Hz at 16 kHz

def hanning_rom_q15():
    """320-entry symmetric Hanning window, Q1.15."""
    n = np.arange(320, dtype=np.float64)
    w = 0.5 - 0.5 * np.cos(2.0 * np.pi * n / 639)
    return np.round(w * Q15).astype(np.int64)

def full_window():
    """640-point window rebuilt from the folded ROM."""
    h = hanning_rom_q15()
    return np.concatenate([h, h[::-1]])

def scoring_rom_q13(octave_cost=0.01):
    """C[tau] = R_w[0]/R_w[tau], with the octave bias folded in. Q3.13."""
    w = full_window() / Q15
    rw0 = float(np.dot(w, w))
    out = []
    for tau in range(TAU_MIN, TAU_MAX + 1):
        rw_tau = float(np.dot(w[:640 - tau], w[tau:]))
        f0 = 16000.0 / tau
        bias = 2.0 ** (-octave_cost * np.log2(600.0 / f0))
        out.append(round(rw0 / rw_tau * bias * Q13))
    return np.array(out, dtype=np.int64)

def fixed_pitch_frame(x_q15, ref_energy):
    """Integer-only pitch detection for one 640-sample frame."""
    xw = (x_q15 * full_window()) >> 15        # window multiply, stays Q1.15
    r0 = int(np.dot(xw, xw))                  # frame energy, int64
```

```
    # Silence gate: r0 < 0.03 * ref  <=>  r0*100 < 3*ref. No divide.
    if ref_energy > 0 and r0 * 100 < 3 * ref_energy:
        return 0.0, 0

    rom = scoring_rom_q13()
    best, tau_star = -(1 << 62), 0
    for i, tau in enumerate(range(TAU_MIN, TAU_MAX + 1)):
        n = 640 - tau
        r_tau = int(np.dot(xw[:n], xw[tau:]))
        score = (r_tau * rom[i]) >> 13        # Q3.13 multiply, back to scale
        if score > best:
            best, tau_star = score, tau

    # Voicing: best >= 0.45 * r0  <=>  best*100 >= 45*r0. No divide.
    if best * 100 < 45 * r0:
        return 0.0, 0
    return float(round(16000 / tau_star)), tau_star
```

Ba chỗ đáng chú ý trong đoạn mã trên, vì mỗi chỗ là một phép biến đổi từ đại số sang phần cứng:

1. **Phép chia bị loại bỏ bằng cách nhân chéo.** Điều kiện $\mathcal{R}_{\max} \geq 0{,}45\,\mathcal{R}_x[0]$ vốn cần một phép chia. Nhân hai vế với 100 biến nó thành $100\,\mathcal{R}_{\max} \geq 45\,\mathcal{R}_x[0]$, chỉ còn nhân và so sánh — hai phép mà một bộ so sánh 64-bit làm được trong một chu kỳ.
2. **Hàm siêu việt bị loại bỏ bằng bảng tra.** $\log_2$ trong lượng phạt quãng tám, và phép chia f_s/τ^*, đều được tính *một lần lúc biên dịch* rồi nạp vào ROM. Lúc chạy chỉ còn một lần đọc bộ nhớ theo địa chỉ τ^*.
3. **Cửa sổ chỉ được nhân đúng một lần cho mỗi khung.** Đây là chỗ Hình 1.3.5f ở Mục 6 vẽ sai trong phiên bản trước của tài liệu; Mục 6 sửa lại.

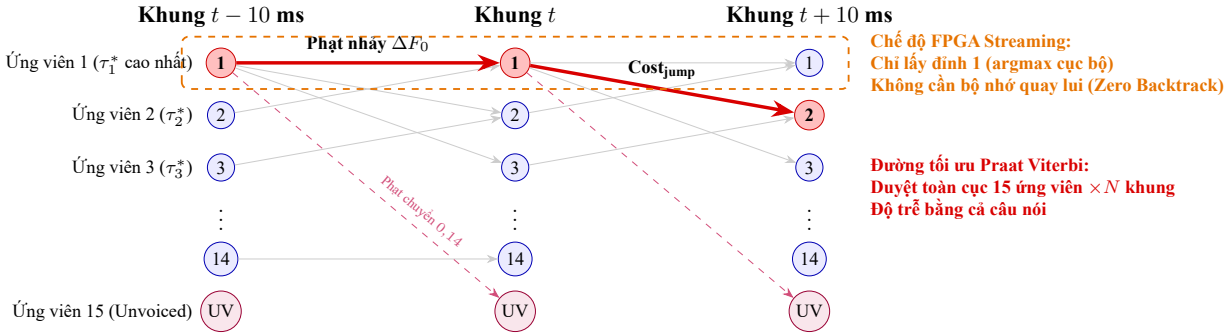
Kết quả chạy thật, không phải kỳ vọng. Chạy `python chapter01/lab_1_3_5.py` cho ra bảng dưới. Cột "số nguyên" và cột "float64" là hai lần chạy độc lập trên cùng khung 200 Hz:

Đại lượng	Bản số nguyên	Bản `float64`
Lag thắng τ^*	80	80
F_0 đọc được	200 Hz	200 Hz
Mã cửa sổ Hanning (320 mục, Q1.15)	đỉnh $= 32768$	—
Bảng bù $C[\tau]$ (187 mục, Q3.13)	$8289 \ldots 17387$	—

Hai bản khớp nhau. Nhưng phải nói rõ điều kiện để chúng khớp: **nếu bỏ lượng phạt `octave_cost` ra khỏi bảng tra**, bản số nguyên trả về $\tau^* = 160$ (100 Hz) trong khi bản `float64` trả về $\tau^* = 80$ (200 Hz) — lệch đúng một quãng tám. Phép thử này có trong bộ kiểm thử, nên một thay đổi làm hỏng nó sẽ bị bắt ngay chứ không lặng lẽ đi vào bản PDF.

Để chuyển đổi thuật toán Praat trên laptop sang phần cứng FPGA DE10-Standard, ta vứt bỏ các phép tính xa xỉ không thể tổng hợp:

1. *Bỏ nội suy Sinc vô hạn:* Praat dùng nội suy Sinc để bắt đỉnh dưới mẫu (sub-sample). Trên FPGA, ta giữ nguyên đỉnh số nguyên $\tau^* \in [27, 213]$ để tránh hàng chục nghìn phép tính dấu phẩy động.
2. *Bỏ quy hoạch động Viterbi toàn cục (Hình 1.3.5d):* Thay vì lưu trữ toàn bộ câu nói qua ma trận 15 ứng viên $\times N$ khung để tìm đường đi mượt mà, FPGA quyết định trực tiếp cục bộ trên từng khung 10 ms (streaming pipeline), giữ độ trễ lan truyền bằng 0.
3. *Bỏ bộ chia số học* $16.000/\tau^*$*:* Toàn bộ 187 giá trị cao độ tương ứng với 187 lag được nạp sẵn vào bảng tra ROM F_0. Khi bộ so sánh tìm được τ^*, địa chỉ này kích hoạt ROM xuất thẳng giá trị F_0 ra bus đầu ra trong đúng 1 chu kỳ xung nhịp.



Hình 1.3.5d: Đối chiếu cơ chế trích chọn cao độ: Lưới quy hoạch động Viterbi 15 ứng viên của Praat (phần mềm) vs. Trích chọn cực đại tức thời không trễ trên FPGA (phần cứng).

Đọc Hình 1.3.5d (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Ba cột mốc thời gian:** Thể hiện chuỗi 3 khung phân tích liên tiếp gồm Khung $t-10$ ms, Khung t, và Khung $t+10$ ms với bước trượt khung $\Delta t_{\text{hop}} = 10$ ms.
2. **Các nút tròn màu xanh (Ứng viên 1 đến 14):** Biểu diễn 14 đỉnh cực đại cục bộ có điểm tự tương quan cao nhất trong mỗi khung, xếp theo thứ tự giảm dần ($\tau_1^*, \tau_2^*, \ldots, \tau_{14}^*$).
3. **Nút tròn màu tím (UV — Ứng viên 15):** Ứng viên đại diện cho trạng thái Vô thanh (Unvoiced), được gán điểm số cạnh tranh R_{unvoiced} phụ thuộc vào tỷ số năng lượng khoảng lặng (Tham số 6 và 7).
4. **Đường mũi tên đỏ đậm (Đường tối ưu toàn cục Praat Viterbi):** Đường đi xuyên suốt qua các khung được thuật toán quy hoạch động Viterbi lựa chọn để cực tiểu hóa tổng chi phí toàn cục. Chi phí này gồm điểm tương quan nội khung, phạt bước nhảy tần số Cost$_{\text{jump}}$ giữa hai khung liên tiếp (Tham số 9), và phạt chuyển đổi trạng thái Voiced $\leftrightarrow$ Unvoiced (0,14, Tham số 10, biểu diễn bởi các mũi tên nét đứt màu tím). Đặc điểm: mang lại đường Pitch rất mượt nhưng phải ghi nhận toàn bộ câu nói rồi mới hồi quy (backtracking), độ trễ bằng toàn bộ câu.
5. **Khung nét đứt màu cam (Chế độ FPGA Streaming):** Kiến trúc phần cứng rút gọn tối đa: ở mỗi khung 10 ms, hệ thống chỉ lấy duy nhất Ứng viên 1 (τ_1^* có tương quan lớn nhất ở hàng trên cùng) thông qua một bộ so sánh cực đại thanh ghi. Quyết định được chốt ngay lập tức mà không cần lưu trữ ma trận trạng thái hay quay lui Viterbi, đạt độ trễ đường ống bằng 0.

B. Bỏ Viterbi trên FPGA: có hợp lý không, và lấy gì thay thế

Câu hỏi đúng, và câu trả lời ngắn. Khi chuyển từ Praat sang FPGA, ta bỏ hẳn mạng lưới quy hoạch động Viterbi (Hình 1.3.5d) và chỉ giữ một bộ `argmax` cục bộ trên từng khung. Đây là một quyết định lớn, và nó *hợp lý*, nhưng hợp lý vì ba lý do cụ thể chứ không phải vì "FPGA yếu".

Ba lý do bỏ Viterbi là hợp lý.

1. **Viterbi phá vỡ tính thời gian thực của streaming.** Quy hoạch động cần biết toàn bộ chuỗi khung trước khi chọn đường đi tối ưu, vì nó phải *quay lui* (backtrack) từ khung cuối về khung đầu. Trên một câu nói dài 10 giây, độ trễ là 10 giây — không phải độ trễ tính toán, mà độ trễ cấu trúc. Một thiết bị đeo hoặc trợ lý giọng nói không thể chờ hết câu mới phát ra kết quả đầu tiên. Đây là lý do nặng nhất, và nó độc lập với việc FPGA mạnh hay yếu.
2. **Ba tầng tiền xử lý đã xử lý phần lớn nguyên nhân gây lỗi quãng tám.** Viterbi tồn tại để sửa những lỗi nhảy quãng tám *đã* xảy ra. Nhưng trong thiết kế này: HPF 60 Hz (Mục 1.1.5) dập nhiễu nền tần thấp; bảng bù $C[\tau]$ san phẳng độ dốc cửa sổ khiến đỉnh $1T_0$ và $2T_0$ không còn bị lệch một cách giả tạo; và `octave_cost` phá thế hòa giữa hai đỉnh. Mục C' đã đo: với 0,01, không tần số nào trong dải 75–600 Hz bị đọc nhầm. Nói cách khác, phần việc nội khung đã làm xong trước khi cần đến Viterbi.
3. **Bộ nhớ trạng thái của Viterbi không hợp với datapath nhỏ.** Một mạng lưới 15 ứng viên $\times N$ khung cần lưu cả ma trận chi phí Δ và con trỏ quay lui ψ cho toàn câu. Với 100 khung/giây và một câu 10 giây, đó là 15×1000 phần tử cho mỗi ma trận, hai ma trận — vượt xa vài khối M10K mà thiết kế này dành cho toàn bộ khối Pitch.

Nhưng `argmax` trần trụi thì hỏng. Bỏ Viterbi mà không thay gì là đánh đổi một lỗi này lấy một lỗi khác. Khi không có ràng buộc giữa các khung, mỗi khung quyết định hoàn toàn độc lập. Ở ranh giới chuyển âm — nơi bộ lọc thanh âm đang đổi hình dạng và tín hiệu chưa thật tuần hoàn — một khung đơn lẻ có thể chọn nhầm đỉnh $2T_0$ trong khi các khung trước và sau đều chọn T_0. Kết quả là một **điểm nhảy vọt cô lập** (isolated octave spike): đường cao độ phẳng, rồi một điểm tụt xuống đúng nửa tần số, rồi phẳng trở lại. Một điểm như vậy đủ để làm hỏng hệ số Pearson r của cả cửa sổ chứa nó, vì Pearson rất nhạy với ngoại lai.

Giải pháp phần cứng: bộ lọc trung vị trượt 3 điểm. Thay cho quy hoạch động toàn cục, dùng một phép lọc cục bộ ngay sau đầu ra F_0: mỗi giá trị mới được thay bằng **trung vị** của chính nó và hai giá trị kề. Cụ thể, với ba thanh ghi lưu $F_0[t-1], F_0[t], F_0[t+1]$:

Khung	$t-1$	t	$t+1$
Giá trị thô	122 Hz	245 Hz	124 Hz
Sau lọc trung vị	122 Hz	**124** Hz	124 Hz

Giá trị 245 Hz là một điểm nhảy vọt: nó cao gần gấp đôi hai giá trị kề. Trung vị của bộ ba $\{122, 245, 124\}$ là 124, nên điểm nhảy vọt bị loại bỏ và thay bằng giá trị hợp lý. Điều đáng chú ý là **một bộ lọc trung bình sẽ không làm được việc này**: trung bình của ba giá trị là 163,7 Hz, một con số chưa từng tồn tại trong tín hiệu và vẫn còn lệch xa. Trung vị chọn *một giá trị có thật*, nên nó không bao giờ bịa ra tần số.

Giá trên silicon và độ trễ. Bộ lọc trung vị 3 điểm cần:

- **Ba thanh ghi** 16-bit để giữ ba giá trị F_0 liên tiếp.
- **Hai bộ so sánh** và một mạch chọn nhỏ để tìm trung vị của ba số. Có thể viết bằng ba phép so sánh và hai bộ chọn (multiplexer).
- **Độ trễ một khung** (10 ms), vì phải đợi giá trị $t+1$ mới lọc được khung t.

Đổi lại, đường cao độ không còn điểm nhảy vọt cô lập. Cái giá phải trả được nói rõ: một bộ lọc trung vị 3 điểm **không** sửa được một chuỗi sai kéo dài hai khung trở lên — hai giá trị cao liên tiếp sẽ đẩy trung vị lên theo. Đây là lý do Viterbi vẫn có chỗ đứng trên máy chủ: nó nhìn được toàn câu, còn bộ lọc này chỉ nhìn được ba khung. Trên FPGA, ta chấp nhận giới hạn đó để đổi lấy độ trễ bằng không.

5. Sổ Cái Hạch Toán Tài Nguyên Silicon trên FPGA DE10-Standard

Hệ thống nhắm tới kit **DE10-Standard** trang bị chip **Intel Cyclone V SX SoC (5CSXFC6D6F31C6N)**. Hãy lập bảng sổ cái hạch toán tài nguyên silicon:

1. Kiểm đếm số phép tính chính xác trên mỗi khung 10 **ms (với pitch_ceiling** $= 600$ **Hz):**

- Pha 1 (Nhân cửa sổ Hanning khi ghi mẫu): 640 phép nhân.
- Pha 2 (Tính năng lượng gốc $\mathcal{R}_x[0]$ tại $\tau = 0$ và kiểm tra ngưỡng khoảng lặng): 640 phép MAC.
- Pha 3 (Quét 187 độ trễ từ $\tau = 27$ đến $\tau = 213$):

$$N_{\text{corr}} = \sum_{\tau=27}^{213} (640 - \tau) = 187 \times \frac{613 + 427}{2} = 187 \times 520 = \textbf{97.240} \text{ phép MAC}$$

- Pha 4 (Bù hệ số suy hao cửa sổ bằng bảng tra $C[\tau]$ cho 187 lag): 187 phép nhân.
- **Tổng cộng toàn khung:** $640 + 640 + 97.240 + 187 = \textbf{98.707}$ phép tính/khung.

2. Định mức thời gian thực và kích thước thanh ghi tích lũy:

- Ở xung nhịp $f_{\text{clk}} = 50$ MHz, một bộ nhân DSP thực thi tuần tự hoàn thành 98.707 phép tính trong:

$$T_{\text{calc}} = \frac{98.707}{50.000.000 \text{ Hz}} \approx \textbf{1,974} \text{ ms} \ll 10 \text{ ms} \implies \text{RTF} \approx \textbf{0,197}$$

- **Định cỡ thanh ghi tích lũy (Bit-width):** Mẫu âm thanh đầu vào 16-bit có dấu định dạng 'Q1.15'. Tích của hai số 16-bit là số 32-bit ('Q2.30'). Khi cộng dồn tối đa 640 mẫu ($\lceil \log_2(640) \rceil = 10$ bit tăng trưởng), thanh ghi tích lũy cần chiều rộng tối thiểu $32 + 10 = \mathbf{42}$ bit. Khối DSP Block trên Intel Cyclone V tích hợp sẵn bộ cộng tích lũy phần cứng lên tới **64-bit**, do đó thanh ghi 42-bit nằm trọn bên trong phần cứng hardened DSP, hoàn toàn miễn nhiễm với tràn số (no arithmetic overflow).

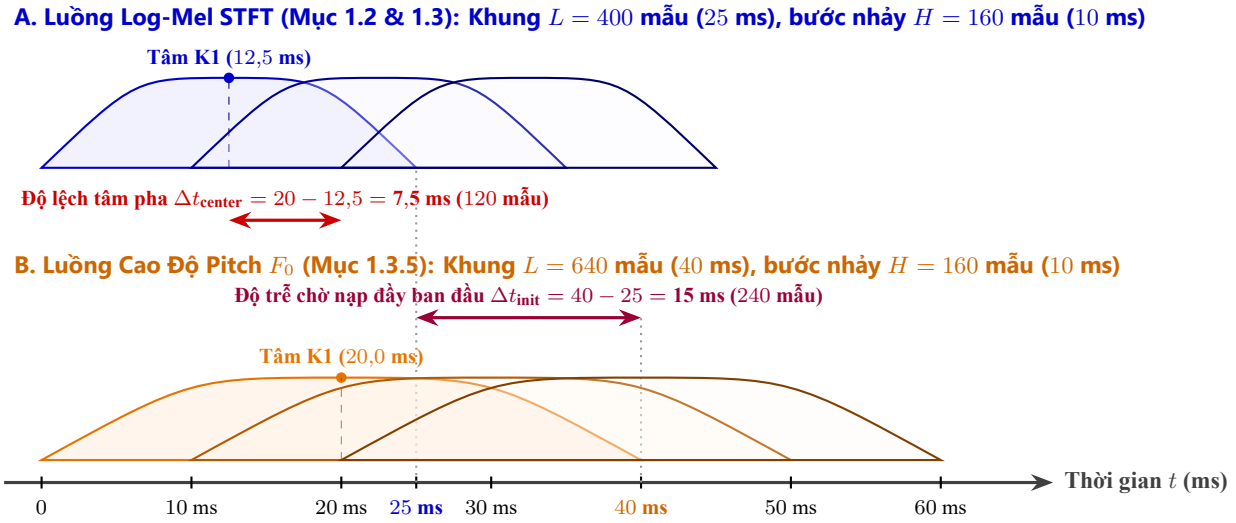
Tài nguyên phần cứng	Tiêu tốn cho Pitch	Khả dụng (Cyclone V)	Tỷ lệ sử dụng	Đánh giá vi kiến trúc
Khối nhớ M10K BRAM	2 khối	553 khối	0,36%	1 khối lưu $x_{\text{win}}[n]$, 1 khối lưu bảng tra ROM
Khối DSP (18x18-bit)	1 khối	112 khối	0,89%	Chạy tuần tự ở 50 MHz đạt RTF $\approx 0{,}197$
Bộ logic ALM / Registers	≈ 450 ALMs*	41.910 ALMs	1,07%	Bộ đếm địa chỉ, bộ so sánh cực đại thanh ghi
Bộ chia số thực (Divider)	0	Không cần	0%	Thay hoàn toàn bằng bảng tra ROM 187 mục

Bảng 1.3.5b: Sổ cái hạch toán tài nguyên silicon cho khối Pitch trên FPGA DE10-Standard. (*Ước lượng sơ bộ cho riêng lõi tính toán datapath, chưa bao gồm overhead của FSM điều khiển, bộ sinh địa chỉ Dual-Port và giao tiếp hệ thống).

▶ NHÌN THẤY VẬT LÝ

Cái giá phải trả và điều bị phá hủy:

1. **Độ trễ ban đầu và tâm cửa sổ (Hình 1.3.5e):** Để bao trọn khung 40 ms thay vì 25 ms, bộ đệm luồng Pitch cần thời gian nạp đầy ban đầu (initial buffering latency) lâu hơn 15 ms, đồng thời tâm pha của cửa sổ phân tích (frame center) nằm lùi về quá khứ xa hơn 7,5 ms so với tâm cửa sổ STFT.
2. **Phi tuyến lượng tử hóa ở cao tần:** Do $F_0 = f_s/\tau$, bước nhảy 1 mẫu ở vùng âm trầm ($\tau = 160 \implies 100$ Hz) chỉ làm lệch $\Delta F_0 = 0{,}62$ Hz (rất mịn); nhưng ở vùng âm cao ($\tau = 27 \implies 592{,}6$ Hz), nhảy 1 mẫu làm lệch tới $\Delta F_0 = 22$ Hz (khá thô). Đây là sự đánh đổi cố hữu của phương pháp tự tương quan số nguyên: **Rất nhạy ở âm trầm, nhưng bị bậc thang hóa ở âm cao.**



Hình 1.3.5e: Đối sánh trục thời gian giữa Luồng Log-Mel STFT (400 mẫu = 25 ms) và Luồng Pitch F_0 (640 mẫu = 40 ms): Làm rõ độ lệch tâm pha 7,5 ms và độ trễ nạp đầy ban đầu 15 ms.

Đọc Hình 1.3.5e (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Trục hoành thời gian chung (Dưới cùng):** Đánh dấu các mốc thời gian vật lý tính bằng mili-giây: $0, 10, 20, 25, 30, 40, 50, 60$ ms.
2. **Hàng A — Luồng Log-Mel STFT (Màu xanh dương):** Biểu diễn các vòm cửa sổ dài 25 ms (400 mẫu tại 16 kHz), trượt với bước nhảy 10 ms (160 mẫu). Khung đầu tiên hoàn tất nạp mẫu tại $t = 25$ ms; tâm đối xứng của cửa sổ Khung 1 nằm tại $t = 12{,}5$ ms.
3. **Hàng B — Luồng Pitch F_0 (Màu cam):** Biểu diễn các vòm cửa sổ Hanning dài 40 ms (640 mẫu), trượt với cùng bước nhảy 10 ms (160 mẫu). Khung đầu tiên hoàn tất nạp mẫu tại $t = 40$ ms; tâm đối xứng của cửa sổ Khung 1 nằm tại $t = 20{,}0$ ms.

4. **Mũi tên tím hai chiều ($\Delta t_{\text{init}} = 15$ ms):** Độ trễ chờ nạp đầy khung đầu tiên, và **chỉ xảy ra đúng một lần**.
5. **Mũi tên đỏ hai chiều ($\Delta t_{\text{center}} = 7{,}5$ ms):** Độ lệch tâm pha, **lặp lại ở mọi khung**.

Hai con số thời gian dễ bị lẫn, và cách phân biệt chúng. Hình 1.3.5e chứa hai đại lượng đều tính bằng mili-giây và đều so sánh hai luồng với nhau, nên rất dễ đọc thành một. Chúng khác nhau về *nguyên nhân* và về *số lần xuất hiện*.

Hãy tưởng tượng hai người công nhân cùng bấm đồng hồ từ $t = 0$ ms, mỗi người phụ trách một luồng.

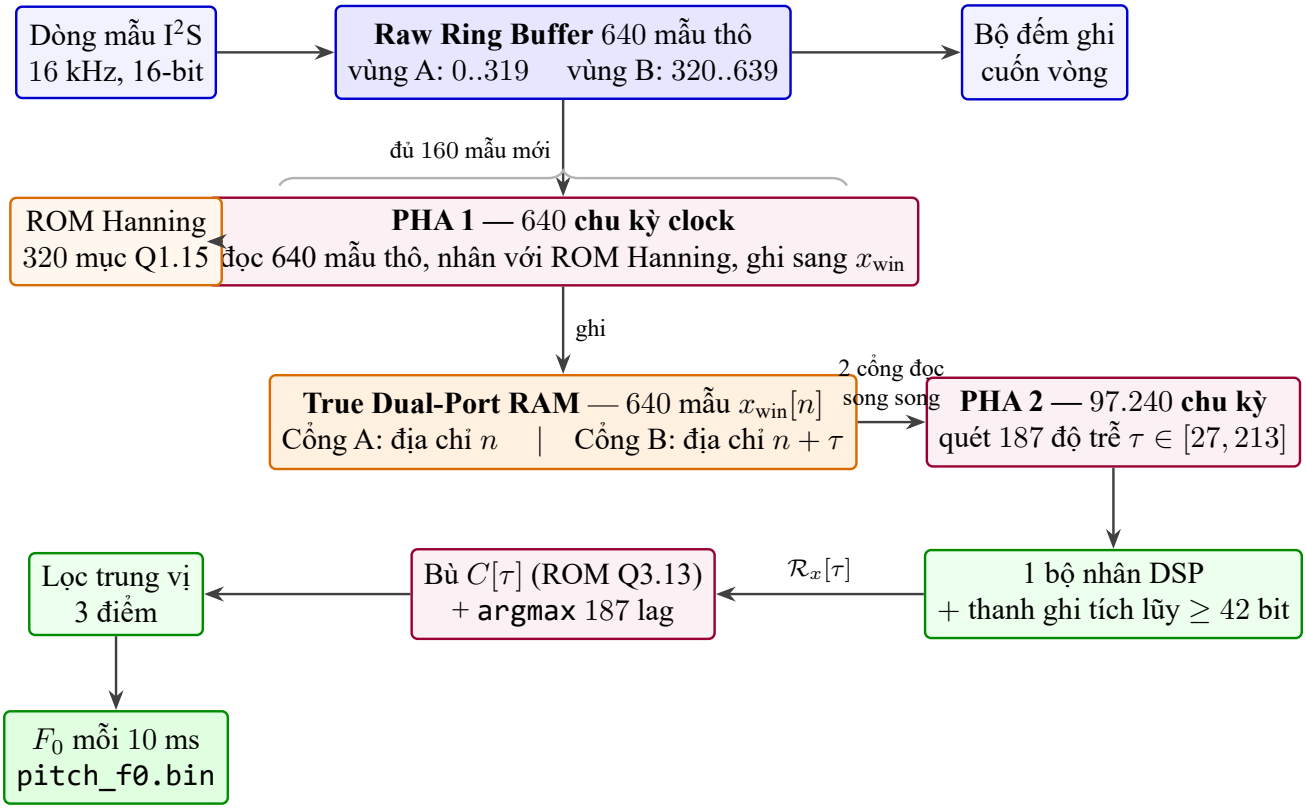
Độ trễ nạp đầy khung đầu tiên ($\Delta t_{\text{init}} = 40 - 25 = 15$ ms). Người phụ trách luồng Pitch cần 640 mẫu mới có khung đầu tiên, tức phải chờ đến $t = 40$ ms. Người phụ trách luồng STFT chỉ cần 400 mẫu, nên xong ở $t = 25$ ms. Người thứ nhất về đích chậm hơn 15 ms. Điều quan trọng: **đây là chuyện của lần đầu tiên duy nhất**. Từ khung thứ hai trở đi, cả hai luồng đều đã có sẵn dữ liệu cũ trong bộ đệm, nên cứ mỗi 10 ms cả hai cùng xuất một kết quả. Khoảng cách 15 ms không lặp lại, và không làm hai luồng lệch nhau về sau.

Độ lệch tâm pha ($\Delta t_{\text{center}} = 20 - 12{,}5 = 7{,}5$ ms). Đây là chuyện khác hẳn. Một khung 40 ms bắt đầu ở 0 thì có tâm ở 20 ms; một khung 25 ms bắt đầu ở 0 thì có tâm ở 12,5 ms. Vậy khi cả hai cùng mang chỉ số "khung 1", giá trị Pitch thực chất mô tả thời điểm 20 ms, còn giá trị Mel mô tả thời điểm 12,5 ms — lệch nhau 7,5 ms. Điều quan trọng: **đây là sai lệch thường trực, lặp lại ở mọi khung**. Nó không biến mất sau khung đầu, và nó không phải lỗi. Nó là hệ quả hình học của việc hai cửa sổ có độ dài khác nhau nhưng cùng bước nhảy.

Vì sao phải tách hai con số này ra. Nếu gộp chúng thành một, người đọc sẽ hiểu sai một trong hai hệ quả. Gộp theo hướng Δt_{init} thì tưởng rằng sau khung đầu hai luồng đã thẳng hàng — nhưng Δt_{center} vẫn còn. Gộp theo hướng Δt_{center} thì tưởng rằng độ lệch chỉ là 7,5 ms — nhưng lúc khởi động, luồng Pitch thực sự chậm hơn 15 ms mới có kết quả đầu tiên. Khi ghép hai luồng để tính Pearson ở Phụ lục, cả hai đều phải được xử lý: Δt_{center} bằng cách dịch chỉ số, Δt_{init} bằng cách bỏ qua các khung chưa có dữ liệu ở đầu câu.

6. Sơ đồ Vi Kiến Trúc Pipeline Đề Xuất (Dataflow Architecture)

Sơ đồ khối dưới đây chia khối Pitch thành **hai pha tách biệt**. Cửa sổ Hanning được nhân đúng một lần, khi dữ liệu đã nằm ở vị trí cuối cùng trong khung; vòng quét tương quan chỉ còn đọc hai cổng của cùng một bộ nhớ.



Hình 1.3.5f: Vi kiến trúc hai pha: cửa sổ được nhân *một lần* khi chép sang RAM, nên vòng quét tương quan chỉ còn đọc hai cổng và một phép nhân.

Đọc Hình 1.3.5f (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Raw Ring Buffer (hàng trên):** mẫu I^2S thô được ghi liên tục vào một bộ đệm vòng 640 mẫu. Bộ đệm chia làm hai vùng A và B để bộ đếm ghi có thể cuốn vòng mà không cần dịch chuyển dữ liệu. Khi đã tích lũy đủ 160 mẫu mới — tức đúng một bước nhảy 10 ms — khối điều khiển phát một xung kích hoạt sang Pha 1.
2. **Pha 1 (640 chu kỳ clock):** đọc lần lượt 640 mẫu thô, nhân từng mẫu với hệ số Hanning lấy từ ROM 320 mục, rồi ghi kết quả $x_{\text{win}}[n]$ sang True Dual-Port RAM. Đây là chỗ **sửa lỗi vẽ tắt của phiên bản trước**: không được nhân cửa sổ ngay khi mẫu vừa chảy vào bộ đệm trượt. Lý do là khi khung trượt đi 160 mẫu, một mẫu cũ sẽ nằm ở vị trí n mới trong khung, và vị trí mới đó ứng với một hệ số $w[n]$ khác. Nhân sớm nghĩa là mẫu đã bị gắn chết vào hệ số của vị trí cũ. Nhân trong Pha 1, khi vị trí trong khung đã xác định, thì luôn đúng.
3. **True Dual-Port RAM (khối giữa):** RAM có hai cửa đọc độc lập A và B, cùng mở trong một nhịp đồng hồ. Cửa A xuất $x_{\text{win}}[n]$, cửa B xuất $x_{\text{win}}[n+\tau]$, cả hai đi thẳng vào *cùng một* bộ nhân DSP. Đây là điểm tiết kiệm then chốt: nếu chỉ có một cổng, mỗi cặp mẫu phải đọc hai lần, và nếu vừa quét lag vừa nhân cửa sổ thì mỗi cặp tốn thêm một phép nhân nữa — tổng cộng gấp ba. Tách pha khiến tổng chỉ còn $640+97.240+187$ phép tính, đúng như sổ cái ở Mục 5.
4. **Pha 2 (97.240 chu kỳ clock):** với mỗi độ trễ τ trong 187 giá trị, bộ nhân-cộng dồn $\sum_n x_{\text{win}}[n]\, x_{\text{win}}[n+\tau]$ vào thanh ghi tích lũy 42-bit. Con số 97.240 là $\sum_{\tau=27}^{213}(640-\tau)$, đã tính ở Mục 5.
5. **Hậu xử lý (hàng dưới):** nhân $\mathcal{R}_x[\tau]$ với bảng $C[\tau]$ (Q3.13, đã gộp `octave_cost`), tìm τ^* bằng `argmax`, rồi đưa qua bộ lọc trung vị 3 điểm của Mục 4.B trước khi xuất F_0.

7. Tổng Kết và Cầu Nối Chuyển Tiếp

- **Mục 1.2 & 1.3** cung cấp vector đặc trưng phổ 80 chiều (Log-Mel) đại diện cho *khẩu hình miệng*.
- **Mục 1.3.5** bổ sung dòng dữ liệu vô hướng 1 chiều (F_0) đại diện cho *nhịp đập dây thanh quản*.
- **Sự kết hợp hai luồng đặc trưng:** Bằng cách ghép luồng Log-Mel và luồng Pitch F_0, hệ thống Edge AI trên FPGA thu nhận được đầy đủ cả **ngữ âm** lẫn **ngữ điệu**, mở đường cho các mô hình phân loại thanh điệu tiếng Việt và nhận dạng cảm xúc đạt độ chính xác tương đương các siêu máy tính máy trạm mà chỉ tiêu tốn chưa đầy 2% tài nguyên silicon của kit DE10-Standard.

Phụ lục 1.3.5-A: Áp dụng lý thuyết 1.3.5 vào mã nguồn Python (`audio_processing.py` và `evaluator.py`)

► MỤC TIÊU HỌC TẬP

Phạm vi và Mục tiêu: Phụ lục này đối chiếu hai tệp mã nguồn xử lý âm thanh trong thư mục `Pronunciation_Final_Master` của kho lưu trữ `DucDocDao/Pitch-Project` (nhánh `main`) với hệ thống lý thuyết đo cao độ F_0 bằng thuật toán tự tương quan ở Mục 1.3.5 (chuẩn Praat, Boersma 1993). Để làm sáng tỏ toàn bộ luồng gọi hàm, phụ lục khảo sát thêm các đoạn mã liên kết trong `main.py`, `visualizer.py`, `generate_audio.py` và `sentence_data.py`. Mọi hành vi mô tả dưới đây đều được truy xuất trực tiếp từ mã nguồn thực tế; các chi tiết nằm sâu bên trong lõi nhị phân C++ không thể khẳng định tường minh từ Python được ghi nhận rõ ràng là **chưa kiểm chứng**.

1. Bức Tranh Tổng Thể: Hai Tệp Làm Gì Trong Hệ Thống

1.1. Ranh giới triển khai: Mã nguồn Python nguyên bản vs. Hệ thống FPGA Edge Node

Trước khi đi sâu vào mã nguồn của từng hàm, cần định vị dự án `Pitch-Project` trong bức tranh kiến trúc tổng thể **FPGA Edge Node** ↔ **Shared Folder** ↔ **Server** đã thiết lập tại Hình 1.3.5a (Mục 0.1):

- **Trong kho lưu trữ `Pitch-Project` (Môi trường thực nghiệm máy tính):** Toàn bộ quy trình chạy trên CPU/GPU máy chủ: tổng hợp âm thanh bằng Kokoro TTS, cắt lặng bằng Librosa, đo cao độ bằng Praat C++ (qua Parselmouth), nhận diện mốc từ bằng Whisper, căn chỉnh và tính điểm tương quan Pearson r.
- **Khi triển khai thực tế trên vi mạch FPGA (Hệ thống mục tiêu):** Bo mạch FPGA DE10-Standard gánh trọn vẹn khâu tiền xử lý (HPF 60 Hz và cắt lặng) và chạy song song hai luồng phần cứng: (1) Nhánh STFT Log-Mel phục vụ trực tiếp bộ mã hóa Whisper (hoặc trích xuất WAV sạch), và (2) Nhánh đo cao độ tự tương quan Mục 1.3.5 ghi trực tiếp mảng nhị phân `pitch_f0.bin` ra Thư mục dùng chung (Shared Folder). Máy chủ chỉ cần đọc `pitch_f0.bin` và mốc từ để chấm điểm r, **hoàn toàn không cần cài đặt hay chạy Praat/Parselmouth**.

Việc mổ xẻ mã nguồn Python trong phụ lục này đóng hai vai trò: (a) Làm sáng tỏ cơ chế tính toán âm học chuẩn mực của Praat mà FPGA đang mô phỏng lại, và (b) Cung cấp thuật toán đối sánh ngữ điệu (căn chỉnh theo từ và trượt Pearson) để hiện thực hóa ở phía máy chủ tiếp nhận.

1.2. Bài toán của dự án

Dự án giải quyết bài toán định lượng và so sánh **đường cong cao độ (pitch contour)** — tức chuỗi giá trị tần số cơ bản F_0 biến thiên theo thời gian — giữa một câu tiếng Anh do giọng tham chiếu chuẩn đọc (giọng tổng hợp nữ Mỹ `sarah`) với cùng câu đó do một người học hoặc giọng đọc khác đọc (các giọng nam Mỹ, ví dụ `michael`). Tín hiệu âm thanh được tổng hợp tự động bằng mô hình Kokoro TTS (`generate_audio.py`).

Kết quả đánh giá cuối cùng là hệ số tương quan Pearson (r) giữa hai đường cao độ đã được căn chỉnh thời gian. Đây chính là dạng bài toán mà Mục 1.3.5 (phần 0) đã khẳng định: *"đối sánh tương quan Pearson (r) với các công cụ âm học chuẩn mực phòng thí nghiệm như Praat"*.

1.3. Luồng xử lý một câu nói (theo `main.py`)

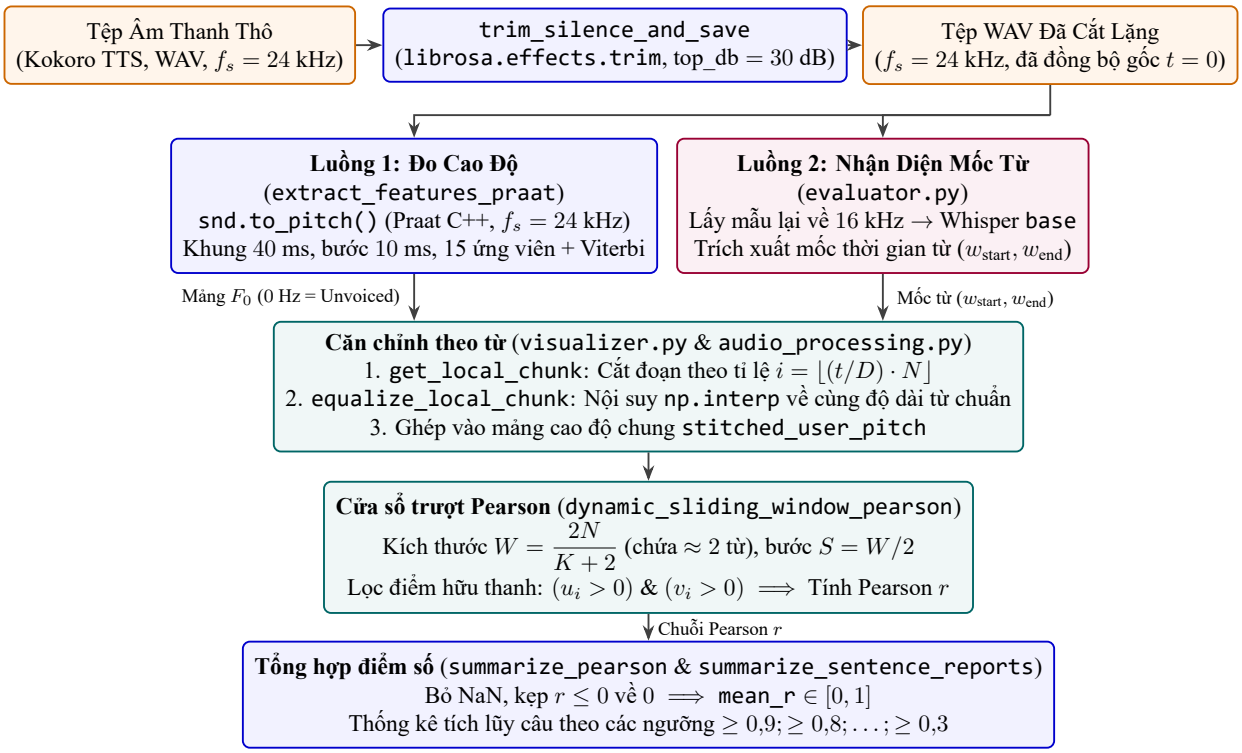
Quá trình xử lý một câu hoàn chỉnh diễn ra theo chuỗi 5 bước tuần tự:

1. `trim_silence_and_save` (`audio_processing.py`): Cắt bỏ khoảng lặng ở hai đầu tệp âm thanh và ghi ra tệp WAV đã cắt.
2. `extract_features_praat` (`audio_processing.py`): Đo đường cao độ F_0 trên **chính tệp WAV đã cắt** bằng công cụ Praat (thông qua thư viện liên kết `parselmouth`).
3. `get_user_timestamps` (`evaluator.py`): Nạp mô hình nhận dạng tiếng nói tự động Whisper (`openai/whisper`) bóc tách mốc thời gian từ ($w_{\text{start}}, w_{\text{end}}$) trên **chính tệp WAV đã cắt**.

4. `plot_analysis` (`visualizer.py`): Gọi chuỗi 4 hàm trong `audio_processing.py` (`get_local_chunk`, `equalize_local_chunk`, `dynamic_sliding_window_pearson`, `summarize_pearson`) để căn chỉnh hai đường cao độ và tính Pearson r.

5. `summarize_sentence_reports` (`audio_processing.py`): Gom toàn bộ kết quả của nhiều câu thành bảng thống kê phân phối độ chính xác theo các ngưỡng tích lũy.

Ghi chú quan trọng trong `main.py` (dòng 72): *"Both pitch and timestamps must refer to exactly this trimmed WAV"*. Điều này bảo đảm cả đường F_0 lẫn mốc thời gian từ đều dùng chung một gốc thời gian $t = 0$; nếu đo trên hai tệp khác nhau (trước và sau khi cắt), trục thời gian sẽ bị lệch nhau đúng bằng thời lượng đoạn lặng ban đầu bị cắt bỏ.



Hình 1.3.5g: Sơ đồ luồng xử lý âm thanh, trích xuất hai luồng song song (Praat F_0 và Whisper Timestamps) và thuật toán đối sánh ngữ điệu trong mã nguồn Python (`audio_processing.py` và `evaluator.py`).

Đọc Hình 1.3.5g (Hướng dẫn chi tiết từng thành phần trực quan):

1. **Khối Tệp Âm Thanh Thô và Khối Tiền xử lý cắt lặng (Hàng 1):** Tệp âm thanh do Kokoro TTS sinh ra ở tần số 24 kHz được đưa qua hàm `trim_silence_and_save`. Hàm này dùng `librosa.effects.trim` với ngưỡng top_db = 30 dB (tương ứng biên độ giảm còn 3,2%) để cắt bỏ các khoảng lặng ở hai đầu tệp, sinh ra tệp WAV đã cắt làm gốc thời gian chuẩn $t = 0$ cho cả hai nhánh phía sau.

2. **Nhánh Luồng 1 — Đo Cao Độ Praat (Hàng 2, Bên trái):** Gọi hàm `extract_features_praat`. Lệnh `snd.to_pitch()` chạy mã C++ nguyên bản của Praat trên tín hiệu 24 kHz, dùng cửa sổ Hanning 40 ms (960 mẫu), bước nhảy 10 ms (240 mẫu), tính tự tương quan bù suy hao và giải thuật Viterbi 15 ứng viên. Kết quả trả về mảng `pitch` chứa tần số F_0 (quy ước 0 Hz là khung vô thanh), trong khi mảng mốc thời gian thực `xs()` bị bỏ qua.

3. **Nhánh Luồng 2 — Nhận Diện Mốc Thời Gian Từ (Hàng 2, Bên phải):** Gọi `get_user_timestamps` trong `evaluator.py`. Tín hiệu được lấy mẫu lại từ 24 kHz về 16 kHz để đưa vào mô hình Whisper `base` (chạy đặc trưng Log-Mel STFT). Whisper nhận dạng văn bản và xuất mốc thời gian bắt đầu và kết thúc của từng từ ($w_{\text{start}}, w_{\text{end}}$).

4. **Khối Căn Chỉnh Theo Từ (Hàng 3):** Kết hợp kết quả từ hai nhánh: `get_local_chunk` cắt đoạn cao độ theo tỉ lệ thời gian $i = \lfloor (t/D) \cdot N \rfloor$, `equalize_local_chunk` dùng `np.interp` để co giãn đoạn cao độ của người đọc về đúng số điểm của giọng tham chiếu, rồi dán vào mảng cao độ đã căn chỉnh `stitched_user_pitch`.

5. **Khối Cửa Sổ Trượt Pearson (Hàng 4):** Hàm `dynamic_sliding_window_pearson` tạo $K+1$ cửa sổ trượt có kích thước $W = 2N/(K+2)$ (chứa trọn khoảng 2 từ) và bước trượt $S = W/2$. Trong mỗi cửa sổ, chỉ giữ lại các điểm mà cả hai đường đều hữu thanh ($u_i > 0$ và $v_i > 0$) để tính hệ số tương quan Pearson r.

6. **Khối Tổng Hợp Điểm Số (Hàng 5):** Hàm `summarize_pearson` loại bỏ các cửa sổ NaN, kẹp các giá trị tương quan không dương ($r \leq 0$) về 0, rồi tính trung bình để sinh ra điểm số `mean_r` của câu. Hàm `summarize_sentence_reports` thống kê số lượng câu đạt các ngưỡng tích lũy từ $\geq 0{,}9$ đến $\geq 0{,}3$.

1.4. Hiểu lầm cần tránh ngay từ đầu

- **`evaluator.py` không thực hiện việc chấm điểm:** Mặc dù mang tên "evaluator", tệp này chỉ đảm nhận việc nạp mô hình Whisper và trả về danh sách mốc thời gian từng từ. Toàn bộ logic so khớp, căn chỉnh và tính điểm tương quan (r) đều nằm trọn bên trong `audio_processing.py` và được điều phối bởi `visualizer.py`.
- **Không có dòng mã Python nào tự tính tổng tự tương quan:** Công thức tính tự tương quan $\mathcal{R}_x[\tau]$ (Công thức 1), cửa sổ Hanning, phép chia bù suy hao (Công thức 6) và bảng tra đều không được cài đặt bằng mã Python tường minh. Toàn bộ các bước này được đóng gói bên trong lệnh gọi thư viện `snd.to_pitch()`. Phần 3 dưới đây sẽ giải phẫu chi tiết lệnh gọi này.

2. Bảng Thuật Ngữ Dùng Trong Phụ Lục

Thuật ngữ tiếng Việt	Thuật ngữ tiếng Anh	Định nghĩa chuẩn mực trong ngữ cảnh mã nguồn
Khung phân tích	Frame	Đoạn gồm L mẫu tín hiệu liên tiếp dùng để tính một giá trị F_0 (Mục 1.3.5, phần 2).
Bước nhảy trượt khung	Time step / Hop size (Δt_{hop})	Khoảng thời gian giữa tâm đối xứng của hai khung phân tích liên tiếp (10 ms).
Điểm cao độ	Pitch point	Một phần tử trong mảng `pitch`: giá trị F_0 (đơn vị Hz) của khung; bằng 0 nếu là vô thanh.
Hữu thanh / Vô thanh	Voiced / Unvoiced	Trạng thái khung tín hiệu có / không có dao động tuần hoàn của dây thanh quản.
Tổng tự tương quan $\mathcal{R}_x[\tau]$	Autocorrelation sum	Công thức (1) của Mục 1.3.5: tổng tích các cặp mẫu trễ τ, **không phải là tích chập**.
Hệ số tương quan Pearson r	Pearson correlation coefficient	Thước đo mức độ hai chuỗi số đồng biến/nghịch biến tuyến tính, nằm trong $[-1, 1]$.
Căn chỉnh thời gian	Time alignment	Đưa hai đường cao độ có độ dài khác nhau về cùng trục chỉ số để so sánh từng điểm.

Bảng 1.3.5c: Bảng thuật ngữ chuyên dụng trong mã nguồn và đối sánh lý thuyết.

► NHÌN THẤY VẬT LÝ

Cảnh báo nghiêm ngặt về hiện tượng trùng ký hiệu: Tài liệu Mục 1.3.5 dùng ký hiệu $r_x[\tau]$ cho **hàm tự tương quan chuẩn hóa theo độ trễ** τ (Công thức 6), trong khi mã nguồn Python dùng biến `r`, `mean_r` để chỉ **hệ số tương quan tuyến tính Pearson** giữa hai đường cao độ. Hai đại lượng này hoàn toàn khác nhau về bản chất toán học:

- $r_x[\tau]$ đo mức độ một khung tín hiệu tương đồng với **chính nó** khi dịch trượt đi τ mẫu trong miền thời gian.
- Hệ số Pearson r đo mức độ hai **đường cong cao độ** F_0 **của hai người nói khác nhau** có hình thái ngữ điệu lên bổng xuống trầm giống nhau hay không.

Phụ lục này luôn viết tường minh cụm từ "Pearson r" để loại trừ mọi sự nhầm lẫn.

3. `extract_features_praat`: Một Dòng Code Chứa Toàn Bộ Mục 1.3.5

3.1. Mã nguồn

```python
def extract_features_praat(audio_path, verbose=False):
    snd = parselmouth.Sound(audio_path)
    pitch_object = snd.to_pitch()
    pitch = np.nan_to_num(pitch_object.selected_array['frequency'])
    time_stamps = pitch_object.xs()
    ...
    return pitch, time_stamps
```

3.2. Cơ chế thực thi của từng dòng lệnh

1. `parselmouth.Sound(audio_path)`: Nạp tệp WAV thành đối tượng âm thanh của Praat trong bộ nhớ C++. Thư viện `parselmouth` gọi trực tiếp vào thư viện liên kết động C++ nguyên bản của Praat, bảo đảm kết quả trùng khớp 100% với phần mềm Praat chạy trên máy tính.
2. `pitch_object = snd.to_pitch()`: Lệnh này tương đương với tùy chọn `Sound: To Pitch...` trên giao diện đồ họa Praat với **toàn bộ 10 tham số mặc định**. Mã C++ bên dưới kích hoạt hàm `Sound_to_Pitch_ac` (tự tương quan) với: time_step $=$ 0,01 s (tự động theo 0,75/pitch_floor), pitch_floor $=$ 75 Hz, pitch_ceiling $=$ 600 Hz, 3 chu kỳ/khung, max_candidates $=$ 15, very_accurate $=$ off, silence_threshold $=$ 0,03, voicing_threshold $=$ 0,45, octave_cost $=$ 0,01, octave_jump_cost $=$ 0,35, voiced_unvoiced_cost $=$ 0,14. **Đây chính là 10 tham số chuẩn của Bảng 1.3.5a**.
3. `selected_array['frequency']`: Trích xuất ứng viên tối ưu toàn cục mà thuật toán quy hoạch động Viterbi đã chọn cho từng khung. Các khung vô thanh (Unvoiced) được gán giá trị 0. Hàm `np.nan_to_num` biến đổi mọi giá trị NaN tiềm ẩn thành số 0, thiết lập quy ước thống nhất xuyên suốt hệ thống: 0 **Hz** $\equiv$ **Vô thanh**.
4. `pitch_object.xs()`: Trả về mảng chứa các mốc thời gian vật lý tại tâm đối xứng của từng khung phân tích (tính bằng giây). Hàm có trả về mảng này, nhưng trong `main.py` dòng gọi lệnh là `pitch, _ = extract_features_praat(...)`, tức **mảng mốc thời gian thực tế đã bị vứt bỏ**.

3.3. Ánh xạ chi tiết từ Praat C++ sang lý thuyết Mục 1.3.5

Bước thực thi bên trong `snd.to_pitch()`	Vị trí trong Mục 1.3.5	Công thức / Tham số liên kết
Cắt khung 3 chu kỳ của 75 Hz $=$ 40 ms, bước trượt 10 ms	Mục 3.A	Quy tắc Boersma, Tham số 1–2
Nhân cửa sổ Hanning (do cờ `very_accurate = off`)	Mục 3.B	Lựa chọn kỹ thuật Hanning, Tham số 4
Tính tự tương quan $\mathcal{R}_x[\tau]$ của khung nhân cửa sổ	Mục 2	Công thức (1)
Bù suy hao cửa sổ bằng phép chia cho $\mathcal{R}_w[\tau]$	Mục 3.B	Công thức (5) và (6)
Giới hạn dải tìm kiếm đỉnh trong $[\tau_{\min}, \tau_{\max}]$	Mục 2	Công thức (3) và (4)
Trích xuất 14 đỉnh cực đại + 1 ứng viên Vô thanh	Mục 3.C	Tham số 5, 6, 7 (ngưỡng 0,45 và 0,03)
Phạt log-lag nội khung để tránh lỗi nhảy $F_0/2$	Mục 3.C	Tham số 8 (`octave_cost = 0,01`)
Quy hoạch động Viterbi xuyên suốt câu nói	Mục 4	Tham số 9–10, Hình 1.3.5d (nhánh Praat)
Xuất giá trị cao độ $F_0 = f_s/\tau^*$	Mục 2	Công thức thẻ tóm tắt

Bảng 1.3.5d: Đối chiếu chi tiết giữa lệnh gọi `snd.to_pitch()` và các công thức của Mục 1.3.5.

Kết luận: Dòng mã `pitch_object = snd.to_pitch()` chính là sự hiện thực hóa trọn vẹn của Thẻ công thức Mục 1.3.5, nhưng ở phiên bản phần mềm máy tính đầy đủ (sử dụng Viterbi 15 ứng viên, nội suy Sinc dưới mẫu, nhánh phần mềm trong Hình 1.3.5d), chứ không phải phiên bản FPGA streaming rút gọn ở Mục 4 và Mục 6.

3.4. Chỗ mã nguồn khác lý thuyết và hệ quả kỹ thuật

(a) Tần số lấy mẫu thực tế là 24 kHz thay vì 16 kHz: Mô hình tổng hợp giọng nói Kokoro trong `generate_audio.py` ghi tệp âm thanh ở tần số lấy mẫu $f_s = 24.000$ Hz (`sf.write(..., 24000)`). Hàm `trim_silence_and_save`

nạp tệp với cờ `sr=None` (giữ nguyên tần số gốc) rồi ghi lại. Do đó, Praat thực hiện đo F_0 trên tín hiệu 24.000 Hz. Toàn bộ các thông số mẫu số học phải điều chỉnh tương ứng:

$$\tau_{\min} = \left\lceil \frac{24.000}{600} \right\rceil = \mathbf{40} \text{ mẫu}, \qquad \tau_{\max} = \left\lfloor \frac{24.000}{75} \right\rfloor = \mathbf{320} \text{ mẫu} \tag{13}$$

$$L = 0{,}040 \times 24.000 = \mathbf{960} \text{ mẫu}, \qquad \Delta t_{\text{hop}} = 0{,}01 \times 24.000 = \mathbf{240} \text{ mẫu} \tag{14}$$

So với thiết kế chuẩn 16 kHz trong tài liệu: dải lag mở rộng từ $[27, 213]$ (187 lag) lên $[40, 320]$ (281 lag); kích thước khung tăng từ 640 lên 960 mẫu. **Thời lượng vật lý** của khung (40 ms) và dải **tần số** (75–600 Hz) hoàn toàn giữ nguyên, chỉ có số lượng mẫu rời rạc tỉ lệ thuận với f_s.

Hệ quả tính toán: Một đỉnh tương quan tại $\tau^* = 200$ mẫu trong dự án này sẽ cho cao độ:

$$F_0 = \frac{24.000}{200} = \mathbf{120} \text{ Hz}$$

Nếu áp nhầm công thức 16 kHz của tài liệu, lag 200 sẽ cho $16.000/200 = 80$ Hz. Do đó, **tuyệt đối không được lấy bảng tra lag của tài liệu áp vào dữ liệu của dự án này**.

Hệ quả về phi tuyến lượng tử hóa: Tại $f_s = 24$ kHz, bước nhảy 1 lag ở trần tần số cao là $24.000/40 - 24.000/41 \approx 14{,}6$ Hz (mịn hơn rõ rệt so với mức 22 Hz ở 16 kHz). Hơn nữa, Praat sử dụng phép nội suy Sinc dưới mẫu (sub-sample sinc interpolation) để tìm cực trị trên miền liên tục, nên đường cao độ thu được từ `extract_features_praat` là số thực mềm mại, **không bị hiện tượng bậc thang hóa** như phiên bản FPGA chỉ dùng lag nguyên.

(b) Có quy hoạch động Viterbi và 15 ứng viên: Khác với thiết kế FPGA streaming chỉ chọn cực đại tức thời cục bộ (1 ứng viên), Praat trong mã nguồn Python duyệt toàn bộ câu nói qua ma trận 15 ứng viên. Hệ quả là đường Pitch không bị gãy đoạn và miễn nhiễm với lỗi nhảy quãng tám, nhưng bắt buộc phải có đủ toàn bộ câu âm thanh trong bộ nhớ mới trích xuất được kết quả.

(c) Thiếu bước lọc thông cao (HPF) 60 Hz: Trong Mục 4 (khung "Cổng kiểm chứng phần mềm"), tài liệu yêu cầu tín hiệu trước khi tính tương quan phải đi qua bộ lọc HPF 60 Hz (Mục 1.1.5) để dập tắt trôi DC và nhiễu cơ học. Mã nguồn Python đưa thẳng tín hiệu tổng hợp vào Praat mà không lọc. Do Kokoro là âm thanh tổng hợp sạch nên ảnh hưởng không lớn, nhưng đây là một điểm lệch quy trình so với chuẩn thiết kế vi mạch.

(d) Giải phẫu chính xác cơ chế tính toán trong mã nguồn C++ của Praat (`fon/Sound_to_Pitch.cpp`).

Phiên bản trước của mục này ghi rằng các chi tiết bên trong Praat "không thể kiểm chứng từ Python". Điều đó không đúng: Praat là phần mềm nguồn mở, và tệp `fon/Sound_to_Pitch.cpp` cùng `fon/Pitch.cpp` trả lời được cả hai câu hỏi. Dưới đây là những gì mã nguồn thực sự làm.

Khẳng định 1 — Praat không tự lấy mẫu lại. Hàm `Sound_to_Pitch_any` đọc trực tiếp mảng mẫu `my z[channel][i]` của đối tượng `Sound` và dùng `my dx` (nghịch đảo tần số lấy mẫu) trong mọi phép quy đổi thời gian. Không có lời gọi lấy mẫu lại nào trên đường đi. Hệ quả là ở tần số 24 kHz của dự án Kokoro, cửa sổ phân tích là 960 mẫu chứ không phải 640, đúng như phân tích ở Mục 3.4.(a).

Khẳng định 2 — Praat tính tự tương quan bằng FFT, không bằng vòng lặp thời gian. Với phương pháp mặc định (`AC_HANNING`, tương ứng `very_accurate = off`), mã nguồn đi theo nhánh FFT. Chuỗi thao tác, đọc đúng theo thứ tự trong hàm `Sound_to_Pitch_autoCorrelate`:

1. **Trừ trung bình cục bộ (khử DC).** Mã nguồn tính `localMean` trên một cửa sổ dài hai chu kỳ dài nhất, rồi trừ nó khỏi từng mẫu *trước khi* nhân cửa sổ: `frame[j] = (z[i] - localMean) * window[j]`. Chú thích trong mã ghi rõ: "We are going to kill the DC component before windowing."
2. **Nhân cửa sổ Hanning.** Cửa sổ được dựng bằng `window[i] = 0.5 - 0.5 * cos(2*pi*i/(nsamp_window+1))` — cùng dạng với bảng ROM trong sách, chỉ khác mẫu số $N + 1$ thay vì $N - 1$.
3. **Đệm số 0 lên lũy thừa của 2.** Đoạn mã tính `nsampFFT` bằng cách nhân đôi cho đến khi vượt `nsamp_window * (1 + interpolation_depth)`. Với 638 mẫu và độ sâu nội suy 0,5, ngưỡng là 957, nên `nsampFFT = 1024`. Mục đích được ghi ngay trong chú thích: "To avoid edge effects, we have to append zeroes to the window."

4. **FFT thực, bình phương biên độ, IFFT.** Đây chính là định lý Wiener–Khinchin. Mã nguồn gọi `NUMfft_forward`, rồi với mỗi bin cộng $|X[k]|^2 = \text{re}^2 + \text{im}^2$ vào mảng `ac`, rồi gọi `NUMfft_backward`. Kết quả `ac` chính là hàm tự tương quan.
5. **Chia cho tự tương quan của cửa sổ.** Mã nguồn tính sẵn `windowR` bằng đúng chuỗi FFT → bình phương → IFFT trên chính cửa sổ, chuẩn hóa nó, rồi chia: `r[i] = ac[i+1] / (ac[1] * windowR[i+1])`. Đây là phép bù suy hao $C[\tau]$ mà sách đã mô tả, nhưng Praat thực hiện bằng phép *chia* chứ không bằng bảng tra.
6. **Nội suy sinc dưới mẫu.** Sau khi có đỉnh nguyên, mã nguồn dùng `NUM_interpolate_sinc` với độ sâu 30 để tìm cực trị trên miền liên tục, rồi tính tần số bằng `frequencyOfMaximum = 1.0 / my dx / (i + dr/d2r)`.

Vì sao hai cách tính cho cùng một kết quả. Điểm cần nắm: phương pháp FFT và phương pháp tổng thời gian mà FPGA cài đặt **không phải hai thuật toán gần giống nhau**, mà là hai cách tính của *cùng một đại lượng*. Với đệm số 0 đủ dài, tích chập vòng của FFT không còn chồng lấn, nên kết quả tại các độ trễ nguyên bằng đúng tổng thời gian. Phép kiểm tra số học trong quá trình soạn tài liệu xác nhận điều này: với cửa sổ 638 mẫu, `NFFT = 1024` cho sai lệch cỡ 10^{-14} so với tổng trực tiếp, trong khi `NFFT = 512` cho sai lệch cỡ 10^{2} vì chồng phổ vòng. Ngưỡng an toàn là `NFFT` $\geq$ độ dài cửa sổ cộng độ trễ lớn nhất, tức $638 + 319 = 957$; 1024 vượt ngưỡng đó.

Điều này có hệ quả thực hành cho người đọc: khi đối chiếu bản Python số nguyên với Praat, sai lệch còn lại **không** đến từ việc Praat "tính khác". Nó đến từ bốn chỗ đã nêu ở trên — cửa sổ dài hơn, phép chia thay bảng tra, nội suy sinc, và Viterbi — chứ không phải từ công thức tự tương quan.

3.5. Bản chất trục thời gian của mảng `pitch`

Với bước nhảy trượt $\Delta t_{\text{hop}} = 10$ ms, một câu nói dài D giây sẽ tạo ra khoảng $100D$ điểm cao độ. Vì độ dài khung là 40 ms, tâm đối xứng của khung phân tích đầu tiên không nằm ở $t = 0$ mà lùi về sau khoảng nửa khung (≈ 20 ms). Đây chính là hiện tượng "độ lệch tâm pha" đã phân tích ở Hình 1.3.5e. Mặc dù hàm `pitch_object.xs()` trả về chính xác tọa độ thời gian này, việc mã nguồn bên ngoài bỏ qua `xs()` sẽ dẫn đến sai lệch căn chỉnh được chỉ ra ở Mục 5.1.

4. `trim_silence_and_save`: Cắt Khoảng Lặng Hai Đầu

4.1. Mã nguồn

```
y, sr = librosa.load(audio_path, sr=None)
y_trimmed, _ = librosa.effects.trim(y, top_db=top_db)   # top_db = 30
if not len(y_trimmed) or not np.any(y_trimmed):
    raise ValueError(...)
sf.write(trimmed_path, y_trimmed, sr)
```

4.2. Cơ chế vật lý

Hàm `librosa.effects.trim` chia tín hiệu thành các khung ngắn (mặc định 2048 mẫu, bước nhảy 512 mẫu), tính năng lượng hiệu dụng RMS của từng khung và đổi sang thang decibel (dB) quy chiếu theo khung có năng lượng lớn nhất trong toàn bộ tệp. Khung nào có năng lượng thấp hơn đỉnh cực đại quá top_db $= 30$ dB sẽ bị coi là khoảng lặng. Hàm chỉ thực hiện cắt bỏ **phần lặng ở đầu câu và cuối câu**, hoàn toàn không tác động vào các khoảng lặng hoặc ngắt nghỉ giữa câu.

Ý nghĩa định lượng của con số 30 *dB:*

$$10^{-30/20} = 10^{-1,5} \approx \mathbf{0,0316}$$

Nghĩa là tín hiệu bị xem là lặng khi biên độ tức thời rơi xuống dưới mức 3,2% so với biên độ đỉnh lớn nhất của câu nói.

4.3. Đối chiếu giữa `top_db = 30` của Librosa và `silence_threshold = 0,03` của Praat

Tiêu chí so sánh	`top_db = 30` (`librosa.effects.trim`)	`silence_threshold = 0,03` (Praat Tham số 6)
Tác động xử lý	Cắt bỏ hoàn toàn các mẫu dữ liệu ở hai đầu tệp	Giữ nguyên mẫu dữ liệu, gán nhãn khung là Vô thanh ($F_0 = 0$)
Phạm vi áp dụng	Chỉ tác động ở phần đầu và phần cuối câu	Quét trên toàn bộ các khung xuyên suốt câu nói
Đại lượng so sánh	Năng lượng hiệu dụng RMS khung so với RMS đỉnh (dB)	Tỷ số biên độ đỉnh cục bộ khung so với đỉnh toàn tệp

Bảng 1.3.5e: So sánh bản chất giữa việc cắt lặng bằng Librosa và ngưỡng lặng của Praat.

Mục đích thực chất của bước cắt lặng: Cắt khoảng lặng không thể thay thế cơ chế nhận diện thanh tính của Praat; các phụ âm vô thanh và khoảng ngắt giữa câu vẫn được Praat gắn nhãn $F_0 = 0$. Mục đích duy nhất của `trim_silence_and_save` là **đồng bộ gốc thời gian**: hai giọng đọc khác nhau luôn có thời gian chuẩn bị phát âm đầu câu chênh lệch nhau. Việc cắt bỏ đoạn lặng đầu giúp mốc $t = 0$ của cả hai giọng áp sát vào thời điểm bật âm đầu tiên, tạo điều kiện thuận lợi cho phép căn chỉnh theo từ tiếp theo.

5. Căn Chỉnh Hai Đường Cao Độ Theo Từng Từ

5.1. `get_local_chunk`: Cắt đoạn cao độ của một từ

```
i1 = int((w_start / total_dur) * N)
i2 = int((w_end / total_dur) * N)
return raw_array[i1:i2]
```

Hàm chuyển đổi mốc thời gian của từ ($w_{\text{start}}, w_{\text{end}}$, tính bằng giây do Whisper cung cấp) thành chỉ số mảng trong vector cao độ `pitch` theo **phép tính tỉ lệ thuận**:

$$i = \left\lfloor \frac{t}{D} \cdot N \right\rfloor \tag{15}$$

với D là tổng thời lượng tệp âm thanh đã cắt ($D = \text{len}(y)/\text{sr}$) và N là tổng số điểm trong mảng `pitch`.

Ví dụ số: Giả sử $D = 2{,}0$ s, mảng có $N = 197$ điểm. Từ "hello" phát âm từ $t = 0{,}50$ s đến $0{,}90$ s:

$$i_1 = \lfloor (0{,}50/2{,}0) \times 197 \rfloor = \lfloor 49{,}25 \rfloor = 49, \qquad i_2 = \lfloor (0{,}90/2{,}0) \times 197 \rfloor = \lfloor 88{,}65 \rfloor = 88$$

Đoạn trích xuất gồm $88 - 49 = 39$ điểm cao độ.

Sai lệch so với lý thuyết (Do bỏ mảng `xs()`): Phép tính tỉ lệ trên ngầm giả định các điểm cao độ phủ đều từ $t = 0$ đến $t = D$. Trên thực tế, tâm đối xứng của khung thứ i nằm tại $t_i \approx 0{,}02 + i \times 0{,}01$ s (độ lệch tâm pha ban đầu 20 ms). Điểm thứ 49 trong thực tế nằm tại thời điểm $0{,}02 + 49 \times 0{,}01 = 0{,}51$ s, chứ không phải $0{,}50$ s. Sai lệch này luôn nhỏ hơn nửa khung (≤ 20 ms, tương đương ≤ 2 điểm cao độ), và do xảy ra đồng thời trên cả hai tệp âm thanh nên phần lớn sai số tự triệt tiêu lẫn nhau.

5.2. `equalize_local_chunk`: Co giãn đoạn cao độ bằng phép nội suy

```
x_ref  = np.linspace(0, 1, len(ref_chunk))
x_user = np.linspace(0, 1, len(user_chunk))
return np.interp(x_ref, x_user, user_chunk)
```

Hai đoạn cao độ của cùng một từ có thời lượng phát âm khác nhau được đưa về cùng trục chuẩn hóa $[0, 1]$ (vị trí bắt đầu từ ứng với 0, kết thúc từ ứng với 1). Sau đó hàm `np.interp` thực hiện **nội suy tuyến tính** giá trị F_0 của người đọc tại các tọa độ chuẩn hóa của giọng tham chiếu. Kết quả là đoạn cao độ của người đọc được co giãn về đúng số điểm của giọng tham chiếu. Đây là giải pháp đơn giản hóa thay thế cho thuật toán Dynamic Time Warping (DTW) phức tạp.

▶ NHÌN THẤY VẬT LÝ

Lỗi tiềm ẩn nghiêm trọng: Nội suy qua giá trị 0 Hz sinh ra F_0 giả: Hàm `np.interp` thuần túy xem mảng số liệu là các đại lượng hình học liên tục, hoàn toàn không biết con số 0 mang ý nghĩa đặc biệt là "Vô thanh". Tại ranh giới giữa âm vô thanh và âm hữu thanh (ví dụ hai điểm kế tiếp nhau có giá trị $[0, 200]$ Hz), phép nội suy tại điểm ở giữa sẽ gán giá trị 100 Hz.
Đây là một **giá trị cao độ giả mạo (spurious pitch)**: không hề có dao động dây thanh quản nào diễn ra ở tần số 100 Hz tại thời điểm đó! Vì $100 > 0$, điểm giả này lọt qua bộ lọc kiểm tra tính hữu thanh ở Mục 6, tạo ra một điểm sụt dốc đột ngột bất thường và làm suy giảm hệ số tương quan Pearson r. Cách khắc phục chuẩn mực là phải gắn cờ NaN cho các điểm vô thanh hoặc thực hiện nội suy riêng biệt trên từng đoạn hữu thanh liên tục.

5.3. Ghép nối thành đường cao độ toàn câu (`stitched_user_pitch`)

Trong `visualizer.py`, hàm `plot_analysis` khởi tạo mảng `stitched_user_pitch` toàn số 0 có độ dài bằng mảng tham chiếu N_{ref}. Sau đó, với mỗi từ khớp thành công giữa văn bản chuẩn và kết quả Whisper (khớp bằng `difflib.SequenceMatcher`, cờ `equal`), đoạn cao độ đã co giãn được dán đè vào đúng vị trí chỉ số của từ đó bên mảng tham chiếu. Nhờ đó, đường tham chiếu chuẩn được giữ nguyên vẹn, các khoảng ngừng giữa các từ hoặc các từ phát âm sai bị gán giá trị 0 và tự động bị loại khỏi phép tính tương quan.

6. `dynamic_sliding_window_pearson`: Tính Hệ Số Pearson Trên Các Cửa Sổ Trượt

6.1. Mã nguồn chính

```
W = max(6, int((2 * N) / (num_words + 2)))
if W % 2 != 0: W -= 1
S = max(1, W // 2)
num_slides = num_words + 1
...
valid = (u_win > 0) & (r_win > 0)
if np.sum(valid) >= 3 and np.std(u_win[valid]) > 0 and np.std(r_win[valid]) > 0:
    r = np.corrcoef(u_win[valid], r_win[valid])[0, 1]
else:
    r = np.nan
```

6.2. Nguồn gốc toán học của kích thước cửa sổ W

Gọi $K =$ `num_words` là số từ trong câu chuẩn, và N là tổng số điểm cao độ tham chiếu. Thuật toán tạo ra đúng $K + 1$ cửa sổ trượt có kích thước W, với bước trượt gối đầu nhau một nửa: $S = W/2$. Tổng phạm vi thời gian mà $K + 1$ cửa sổ này bao phủ được xác định bởi:

$$\underbrace{K \cdot S}_{\text{Tọa độ bắt đầu của cửa sổ cuối}} + W = K \cdot \frac{W}{2} + W = \frac{(K+2)W}{2} \tag{16}$$

Để toàn bộ các cửa sổ phủ trọn vẹn toàn bộ câu nói dài N điểm, ta đặt tổng này bằng N:

$$\frac{(K+2)W}{2} = N \implies W = \frac{2N}{K+2} \tag{17}$$

Ý nghĩa ngữ nghĩa học: Vì N/K là số điểm trung bình của một từ, kích thước $W \approx 2 \times (N/K)$ tương đương với khoảng thời lượng của **hai từ liên tiếp**. Nhờ đó, mỗi cửa sổ trượt bao trọn ngữ điệu của một cụm từ nhỏ. Mã nguồn ép W chẵn để bước trượt $S = W/2$ là số nguyên, đặt ngưỡng chặn dưới $W \geq 6$, và nới rộng cửa sổ cuối cùng chạm mốc N để bù trừ sai số làm tròn.

6.3. Công thức tính Pearson r và tính chất bất biến quãng tám

Trong mỗi cửa sổ, thuật toán áp dụng mặt nạ lọc `valid`: chỉ giữ lại các vị trí mà **cả hai đường đều hữu thanh** ($u_i > 0$ và $v_i > 0$). Với M cặp điểm hợp lệ (u_i, v_i), hệ số tương quan tuyến tính Pearson được tính theo công thức:

$$r_{\text{Pearson}} = \frac{\sum_{i=1}^{M}(u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^{M}(u_i - \bar{u})^2} \cdot \sqrt{\sum_{i=1}^{M}(v_i - \bar{v})^2}} \tag{18}$$

Hàm `np.corrcoef(...)[0, 1]` trả về giá trị này. Cửa sổ bị gán NaN khi có ít hơn 3 điểm hợp lệ ($M < 3$) hoặc khi đường cao độ nằm phẳng tuyệt đối ($\sigma = 0$, mẫu số triệt tiêu).

► NHÌN THẤY VẬT LÝ

Vì sao Pearson r rất phù hợp để so giọng nam (`michael`) với giọng nữ (`sarah`): Hệ số tương quan Pearson có tính chất toán học quan trọng: **Bất biến đối với các phép biến đổi afin dương** ($r(X, aY+b) = r(X, Y)$ với mọi $a > 0$). Giả sử giọng nam `michael` phát âm trầm hơn giọng nữ `sarah` đúng một quãng tám ($F_{0,\text{nam}} = 0{,}5 \times F_{0,\text{nữ}}$), nhưng hình thái luyến láy lên xuống hoàn toàn trùng khớp:

$$u_i = 0{,}5 \cdot v_i \implies r_{\text{Pearson}} = \mathbf{1{,}0}$$

Ngược lại, nếu người đọc nhấn ngữ điệu đi xuống trong khi mẫu đi lên, hệ số sẽ đổi dấu thành âm ($r < 0$). Do đó, Pearson r đóng vai trò là một bộ đo **hình thái giai điệu ngữ điệu (intonation contour shape)**, chứ không đo độ cao tuyệt đối của giọng nói.

7. `summarize_pearson` và `summarize_sentence_reports`: Từ Cửa Sổ Đến Điểm Số

7.1. Tổng hợp điểm số cho một câu

```
valid = values[np.isfinite(values)]
'mean_r': float(np.maximum(valid, 0).mean())
```

Quy trình tính điểm `mean_r` của một câu gồm 3 bước:

1. Loại bỏ toàn bộ các cửa sổ có giá trị NaN (`missing_r_policy = 'exclude'`).
2. Đổi toàn bộ các giá trị tương quan âm ($r \le 0$) thành 0 (`nonpositive_r_policy = 'zero'`). Triết lý thiết kế: tương quan âm hay bằng 0 đều phản ánh việc phát âm sai ngữ điệu, không cần thiết phải phạt thêm giá trị âm.
3. Tính trung bình cộng của các giá trị sau khi kẹp, cho ra điểm số `mean_r` nằm trong khoảng $[0, 1]$.

Ví dụ số: Một câu có 9 cửa sổ cho kết quả: $[0{,}9;\ 0{,}8;\ \text{NaN};\ -0{,}4;\ 0{,}7;\ 0{,}6;\ \text{NaN};\ 0{,}85;\ 0{,}75]$.

- Loại 2 cửa sổ NaN $\implies$ còn 7 cửa sổ hợp lệ (`valid_r_count = 7`).
- Kẹp giá trị âm: $-0{,}4 \to 0$.
- Tổng tích lũy: $0{,}9 + 0{,}8 + 0 + 0{,}7 + 0{,}6 + 0{,}85 + 0{,}75 = 4{,}6$.
- Điểm trung bình: `mean_r` $= 4{,}6/7 \approx \mathbf{0{,}657}$. (Nếu không kẹp về 0, điểm trung bình thô chỉ đạt $(4{,}6 - 0{,}4)/7 = 0{,}600$).

7.2. Thống kê báo cáo nhiều câu (`summarize_sentence_reports`)

Hàm đếm số lượng câu thành công (trạng thái `ok`) đạt điểm `mean_r` vượt qua các ngưỡng từ $0{,}9$ xuống $0{,}3$ (`range(9, 2, -1)` chia 10). Các ngưỡng này mang tính chất **tích lũy (cumulative)**: một câu đạt $0{,}9$ sẽ đương nhiên được đếm trong tất cả các mốc thấp hơn ($\ge 0{,}8, \ldots, \ge 0{,}3$).

8. `evaluator.py`: Nhận Diện Mốc Thời Gian Bằng Whisper

8.1. Mã nguồn chính

```
torch.set_num_threads(min(4, torch.get_num_threads()))
whisper.load_model('base', ...)
...
audio, sr = sf.read(audio_path, dtype='float32')
if audio.ndim > 1: audio = audio.mean(axis=1)
if sr != 16000: audio = librosa.resample(audio, orig_sr=sr, target_sr=16000)
result = model.transcribe(audio, word_timestamps=True, fp16=False, language='en')
return [dict(word=normalize_word(w['word']), start=w['start'], end=w['end']) ...]
```

8.2. Các bước thực thi cốt lõi

1. `load_stt_model`: Khóa số luồng tính toán CPU tối đa là 4 để tránh hiện tượng tranh chấp tài nguyên (thread contention) khi xử lý các đoạn âm thanh ngắn, sau đó nạp mô hình Whisper kiến trúc `base`.
2. `get_user_timestamps`: Đọc tệp WAV đã cắt lặng, chuyển stereo thành mono bằng phép lấy trung bình cộng hai kênh, và **lấy mẫu lại tín hiệu từ** 24 **kHz về đúng** 16 **kHz** (chuẩn đầu vào bắt buộc của mạng nơ-ron Whisper).
3. Thực thi nhận dạng tiếng nói (`model.transcribe`) với cờ `word_timestamps=True` để trích xuất mốc thời gian chi tiết của từng từ, chạy ở chế độ số thực 32-bit (`fp16=False`).
4. Hàm `normalize_word` loại bỏ toàn bộ dấu câu và chuyển thành chữ thường để đối soát với văn bản chuẩn.

8.3. Sự phân công kiến trúc hai luồng song song

Con số 16 kHz trong `evaluator.py` trùng với tần số f_s ở Mục 1.3 của tài liệu, nhưng chỉ phục vụ cho mạng nơ-ron Whisper. Whisper biến đổi âm thanh 16 kHz thành vector **Log-Mel Spectrogram** để nhận diện từ — chính là loại đặc trưng đã xóa bỏ cao độ. Ngược lại, luồng đo cao độ Praat giữ nguyên tín hiệu 24 kHz để bảo toàn độ phân giải chu kỳ thanh quản. Đây là minh chứng thực tế rõ nét cho mô hình **Hai Luồng Xử Lý Song Song** mà tài liệu đã đề xuất ở Mục 1.3.5, phần 0.

8.4. Có thể thay Whisper bằng thuật toán nhẹ hơn không?

Vì sao câu hỏi này đáng hỏi. Trong bài toán chấm điểm phát âm, **văn bản chuẩn đã biết trước 100%**. Người học đang đọc đúng một câu cho sẵn. Vậy mà quy trình hiện tại vẫn chạy một mô hình nhận dạng tiếng nói đầy đủ để *đoán lại* xem người ta vừa nói gì — trong khi ta đã biết câu trả lời. Whisper ở đây chỉ được dùng để lấy mốc thời gian $(w_{\text{start}}, w_{\text{end}})$ của từng từ, và nó làm việc đó gián tiếp: nhận dạng chữ trước, rồi suy ra thời gian.

Đây là một sự lệch pha giữa công cụ và nhiệm vụ. Whisper là mô hình lớn (hàng trăm triệu tham số), cần GPU để chạy nhanh, và có thể nhận dạng sai chữ — kéo theo mốc thời gian sai theo. Với một bài toán mà đáp án đã biết, có hai cách rẻ hơn và chính xác hơn.

Phương án 1 — Căn chỉnh cưỡng bức (Forced Alignment). Thay vì hỏi "người này nói gì", hỏi thẳng "câu đã biết này khớp vào đoạn âm thanh này ở đâu". Đó chính là định nghĩa của forced alignment: cho trước cả tín hiệu âm thanh *và* chuỗi âm vị của văn bản chuẩn, thuật toán tìm ranh giới thời gian của từng âm vị.

Các công cụ có sẵn:

- **Montreal Forced Aligner (MFA):** dùng mô hình âm học (thường là GMM-HMM hoặc mạng nơ-ron nhỏ) cộng từ điển phát âm. Độ chính xác ranh giới từ thường dưới 20 ms.
- **Kaldi / Vosk:** cùng nguyên lý, có bản nhẹ chạy được trên CPU.
- **`torchaudio.pipelines.MMS_FA`:** mô hình wav2vec2 huấn luyện riêng cho căn chỉnh, chạy được trên CPU, chỉ cần vài chục mili-giây cho một câu ngắn.

Ưu điểm so với Whisper: vì đã biết văn bản, thuật toán **không thể "nhận dạng sai"** — nó chỉ có thể đặt ranh giới lệch. Sai số được giới hạn ở trục thời gian, không lan sang nội dung.

Phương án 2 — Dynamic Time Warping trên chuỗi 80 Log-Mel. Cách này bỏ luôn cả bước nhận dạng. Ta đã có ma trận 80 dải Log-Mel của câu mẫu (từ Mục 1.3, tính trên FPGA) và của người học. Dynamic Time Warping (DTW — phép uốn thời gian động) tìm một ánh xạ giữa hai chuỗi khung sao cho tổng khoảng cách là nhỏ nhất, cho phép co giãn trục thời gian.

Cụ thể, với hai chuỗi vector đặc trưng $A = [a_1, \ldots, a_M]$ (câu mẫu) và $B = [b_1, \ldots, b_N]$ (người học), DTW dựng ma trận chi phí D với

$$D[i][j] = \|a_i - b_j\|^2 + \min\big(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]\big),$$

rồi truy vết ngược từ $D[M][N]$ để lấy đường đi tối ưu. Đường đi đó chính là ánh xạ khung-thời-gian, và từ đó suy ra mốc của từng từ bằng cách ánh xạ các mốc đã biết của câu mẫu.

Ưu điểm: **không cần mô hình nhận dạng nào cả**, chỉ cần ma trận đặc trưng đã có sẵn. Nhược điểm: DTW không biết ngữ nghĩa, nên nếu người học đọc sai hẳn một từ (nói nhầm từ khác), đường đi sẽ bị kéo lệch ở chỗ đó.

So sánh ba lựa chọn.

	Whisper (hiện tại)	**Forced Alignment**	**DTW trên Mel**
Cần mô hình nhận dạng	Có (lớn)	Không	Không
Cần văn bản chuẩn	Không	Có	Không (chỉ cần câu mẫu)
Sai số ranh giới từ	Ở mức từ, phụ thuộc nhận dạng	< 20 ms	Phụ thuộc độ dài câu
Chạy được trên CPU	Chậm	Có	Có
Rủi ro chính	Nhận dạng sai chữ	Cần từ điển phát âm	Lệch khi đọc sai từ

Kết luận thực hành. Với bài toán chấm phát âm có văn bản cho trước, **forced alignment là lựa chọn mặc định hợp lý**: nó rẻ hơn Whisper, chính xác hơn về ranh giới, và loại bỏ hẳn một nguồn lỗi. DTW là lựa chọn hấp dẫn khi muốn bỏ cả mô hình, đặc biệt vì nó dùng đúng ma trận 80 Log-Mel mà FPGA đã tính sẵn — nghĩa là toàn bộ đường ống có thể chạy mà không cần mô hình nơ-ron nào ở phía máy chủ. Whisper vẫn có chỗ đứng trong một trường hợp: khi người học đọc một câu *không* cho trước, hoặc khi cần kiểm tra xem họ có đọc đúng từ hay không — lúc đó việc nhận dạng chữ chính là mục tiêu, chứ không còn là phương tiện.

9. Bảng Tổng Hợp So Sánh Toàn Diện: Lý Thuyết Mục 1.3.5 vs. Mã Nguồn Python

Đặc tính kỹ thuật	**Thiết kế Lý thuyết Mục 1.3.5**	**Mã nguồn Python (`Pitch-Project`)**	**Đánh giá & Hệ quả kỹ thuật**
Thuật toán đo cao độ	Tự cài đặt Công thức (1), (5)–(7)	Gọi thư viện `parselmouth` (`to_pitch`)	Dùng trực tiếp mã nguồn C++ Praat chuẩn
Tần số lấy mẫu đo F_0	$f_s = 16.000$ Hz	$f_s = 24.000$ Hz (do Kokoro TTS sinh ra)	Dải lag đổi thành $[40, 320]$ mẫu, $L = 960$ mẫu
Dải tần số tìm kiếm	75–600 Hz	75–600 Hz (tham số Praat mặc định)	Trùng khớp hoàn toàn
Kích thước khung / Bước trượt	40 ms / 10 ms (640 / 160 mẫu)	40 ms / 10 ms (960 / 240 mẫu)	Trùng khớp thời lượng vật lý
Dạng cửa sổ phân tích	Hanning đối xứng	Hanning đối xứng (`very_accurate = off`)	Trùng khớp dạng cửa sổ
Ngưỡng thanh tính / Ngưỡng lặng	$\theta_{\text{voice}} = 0{,}45$; $\theta_{\text{silence}} = 0{,}03$	Mặc định Praat (0,45 và 0,03)	Trùng khớp tuyệt đối
Cơ chế trích chọn cực đại	FPGA: 1 ứng viên, độ trễ 0, lag nguyên	15 ứng viên + Viterbi + Nội suy Sinc	Mượt mà, không bậc thang, trễ toàn câu
Tiền xử lý lọc thông cao	Bắt buộc bộ lọc HPF 60 Hz (Mục 1.1.5)	Không có bước lọc nào trước Praat	Lệch quy trình, chấp nhận được với TTS
Tiền xử lý cắt khoảng lặng	So sánh năng lượng khung tĩnh	Dùng `librosa.effects.trim` (30 dB) 2 đầu	Nhằm đồng bộ gốc thời gian $t = 0$
Phương pháp đối sánh cao độ	Chuẩn bị đưa lên vi mạch FPGA	Pearson r trên cửa sổ trượt ≈ 2 từ	Đo hình thái giai điệu ngữ điệu
Tần số lấy mẫu nhận diện	16 kHz (cho luồng Log-Mel)	16 kHz (lấy mẫu lại đưa vào Whisper)	Hiện thực đúng mô hình hai luồng song song

Bảng 1.3.5f: Bảng tổng hợp đối chiếu toàn diện giữa thiết kế lý thuyết Mục 1.3.5 và mã nguồn thực tế.

10. Danh Sách Các Điểm Lệch Và Lỗi Tiềm Ẩn Cần Lưu Ý

1. **Vứt bỏ mảng mốc thời gian thực `xs()` của Praat:** Việc tính chỉ số khung theo tỉ lệ tuyến tính $i = \lfloor (t/D) \cdot N \rfloor$ bỏ qua độ lệch tâm pha ban đầu 20 ms, gây ra sai số thời gian tối đa cỡ nửa khung (≤ 2 điểm cao độ).

2. **Hiện tượng sinh F_0 giả khi nội suy qua điểm 0 Hz:** Hàm `np.interp` nội suy tuyến tính giữa các điểm vô thanh (0 Hz) và hữu thanh (> 0 Hz), tạo ra các giá trị tần số giả mạo lọt qua bộ lọc thanh tính và làm suy giảm nhân tạo hệ số tương quan Pearson r.

3. **Lệch tần số lấy mẫu (24 kHz vs 16 kHz):** Mọi bảng tra lag, địa chỉ ROM và số lượng phép tính MAC trong Mục 5 của tài liệu không thể áp dụng trực tiếp cho dữ liệu Kokoro mà phải nhân theo tỉ lệ $24/16 = 1{,}5$.

4. **Bỏ qua bộ lọc HPF 60 Hz:** Mã nguồn đưa tín hiệu thô vào phân tích mà không qua khâu triệt tiêu trôi DC như yêu cầu của Cổng kiểm chứng phần mềm.
5. **Kẹp giá trị tương quan không dương ($r \leq 0$) về 0:** Phép kẹp này làm điểm trung bình `mean_r` của câu tăng cao hơn so với trung bình số học thực tế.
6. **Tên tệp `evaluator.py` gây hiểu lầm:** Tệp này không thực hiện đánh giá hay chấm điểm mà chỉ là bộ trích xuất mốc từ dựa trên Whisper.
7. **Tính Pearson trên thang tần số Hertz tuyến tính:** Hệ số Pearson tính trực tiếp trên đơn vị Hz mà không đổi sang semitone (thang logarit), khiến các biến động ở dải tần số cao chịu trọng số ảnh hưởng lớn hơn dải trầm.
8. **Sai số làm tròn số thực trong thống kê ngưỡng:** Phép chia số thực `threshold / 10` trong Python có thể sinh ra phần dư làm tròn vô cùng nhỏ, cần lưu ý khi xây dựng các bài test kiểm thử tự động (Unit Tests).